Analizador Léxico de PHP

Guía completa de estudio para la defensa del proyecto

Proyecto **compilador** — Java 8+, NetBeans (Ant), sin librerías externas ni expresiones regulares

Clase principal: `compilador.Compilador` · Jar: `dist/compilador.jar`

Qué contiene esta guía: la teoría necesaria, la arquitectura, el recorrido secuencial completo del programa (desde `java -jar` hasta el código de salida), la explicación de **cada clase y cada método** con su código real, trazas paso a paso de ejemplos, el catálogo de errores, las decisiones de diseño, escenarios de "¿qué pasa si cambio esto?" y un banco de preguntas del profesor con respuestas modelo.

Todo el código citado se extrajo directamente de los archivos fuente del proyecto; todas las salidas mostradas provienen de ejecuciones reales del jar.

Índice

1. Cómo estudiar con esta guía y guion de 20 minutos 5

Las cinco ideas que debes dejar claras . . . 5

Guion sugerido para la exposición (20 minutos) . . . 5

Comandos que debes saber de memoria . . . 6

2. Conceptos teóricos que el profesor espera escuchar 7

¿Qué es el análisis léxico? . . . 7

Token, lexema y patrón . . . 7

Expresiones regulares y autómatas finitos . . . 7

Coincidencia más larga (maximal munch) . . . 7

Prioridad entre patrones . . . 7

Modos (estados léxicos) . . . 7

Recuperación de errores . . . 8

Tabla de símbolos . . . 8

Las cuatro etapas léxicas que pide el curso . . . 8

3. El problema: requisitos y restricciones 9

Lo que pide el enunciado . . . 9

Restricciones duras (PRD §2) . . . 9

Archivo de entrada de referencia (codigo.php) . . . 9

4. Arquitectura general 11

Paquetes y clases . . . 11

Diagrama del flujo de datos . . . 11

Principios de diseño aplicados . . . 12

5. Recorrido secuencial completo del programa 13

Diagrama de secuencia simplificado . . . 14

6. Paquete app: punto de entrada y orquestación 15

Compilador (clase principal) . . . 15

ExitCode . . . 15

CliOptions . . . 15

Application . . . 16

FileAnalyzer . . . 18

AnalysisResult . . . 18

7. Paquete io: localizar, leer y escribir 19

PhpFileLocator — encontrar el archivo sin ruta fija . . . 19

SourceLine y SourceFile — el archivo en memoria . . . 21

SourceFileReader — leer sin alterar el formato . . . 22

LogFormat — formato de la bitácora . . . 24

ErrorLogWriter — escribir -errors.log . . . 24

8. Etapa 1 — Obtención (adquisición) de tokens **26**

LexerMode . . . 26

CharacterStream — el lector de caracteres . . . 26

Lexer — el bucle principal . . . 27

LexemeInvariant — autocomprobación . . . 29

9. Etapa 2 — Clasificación de tokens **30**

TokenType y TokenCategory . . . 30

Token . . . 30

ReservedWords . . . 31

TokenClassifier . . . 32

10. Etapa 3 — Determinación de patrones (autómatas) **33**

Contrato: LexemePattern y PatternMatch . . . 33

PatternRegistry — el orden de prueba . . . 33

AsciiChars — clases de caracteres . . . 34

Patrones simples . . . 34

Autómatas con enum State . . . 36

OperatorPattern — coincidencia más larga . . . 39

11. Etapa 4 — Validación de lexemas **40**

Infraestructura: Validator, ValidationContext, ValidationPipeline . . . 40

LineText — utilidades de texto . . . 41

OpeningTagValidator — códigos 100 a 106 . . . 41

ClosingTagValidator — códigos 110 a 116 . . . 43

AssignmentTarget y AssignmentTargetExtractor — qué hay a la izquierda del = . . . 44

IdentifierValidator — códigos 200 a 205 . . . 46

CommentValidator — códigos 300 a 304 . . . 48

LexemeValidator — códigos 400 a 402 . . . 49

12. Paquetes de soporte: errors, symbols y report **51**

ErrorCode — el catálogo . . . 51

LexicalError — un error . . . 51

ErrorCollector . . . 52

SymbolTable y SymbolEntry . . . 52

ConsoleReporter . . . 52

13. Pruebas: FixtureRunner y fixtures **54**

Los 16 fixtures . . . 54

identifiers.php: entrada y errores esperados . . . 55

comments.php: entrada y errores . . . 55

14. Trazas paso a paso **56**

Traza 1: tokenizar la línea 19 de codigo.php . . . 56

Traza 2: validar $na@me = 6; . . . 56

Traza 3: $ñame = 1; (deduplicación) . . . 57
Traza 4: comentario que “continúa” en la línea siguiente . . . 57
Traza 5: construcción de la bitácora . . . 57
Traza 6: números . . . 57
15. Catálogo completo de errores **58**
Salida real con varios errores a la vez . . . 59
16. Decisiones de diseño y limitaciones **60**
Limitaciones (fuera de alcance, PRD §10) . . . 60
Detalles finos que conviene conocer (por si te los señalan) . . . 60
17. Escenarios: ¿qué pasa si cambia algo en el código? **61**
18. Preguntas probables del profesor (con respuestas) **64**
19. Hoja de repaso final (memorizar) **69**
En una frase . . . 69
Números clave . . . 69
Orden de patrones (regla mnemotécnica) . . . 69
Los 12 pasos del flujo . . . 69
Frases de cierre para la defensa . . . 69

1. Cómo estudiar con esta guía y guion de 20 minutos

No vas a poder mirar el PDF durante la defensa, así que la guía está pensada para **entender** y luego **memorizar una estructura**. Recomendación de estudio en tres pasadas:

1. **Primera pasada (comprensión):** lee los capítulos 2 a 5 (teoría, requisitos, arquitectura y flujo secuencial). Al terminar deberías poder dibujar de memoria el diagrama de paquetes y recitar los 12 pasos del flujo.
2. **Segunda pasada (detalle):** capítulos 6 a 13, clase por clase. Para cada método pregúntate: *¿qué recibe?, ¿qué devuelve?, ¿quién lo llama?, ¿por qué existe?*
3. **Tercera pasada (defensa):** capítulos 14 a 19: trazas, catálogo de errores, decisiones, escenarios "¿qué pasa si…?", preguntas del profesor y la hoja de repaso final. Practica respondiendo en voz alta, sin leer.

Las cinco ideas que debes dejar claras

1. Es la **fase de análisis léxico** de un compilador para PHP: convierte el texto del archivo en una lista de **tokens** y valida etiquetas, identificadores y comentarios.
2. Está organizado en **cuatro etapas léxicas** en paquetes separados: (1) obtención de tokens, (2) clasificación, (3) determinación de patrones y (4) validación de lexemas.
3. **No usa regex ni generadores** (JFlex, ANTLR…): cada patrón es un **autómata finito escrito a mano** y su Javadoc indica la expresión regular equivalente.
4. **Nunca altera el archivo**: la bitácora `<nombre>-errors.log` es una copia numerada byte a byte idéntica, seguida por la lista de errores específicos ordenados.
5. Está **probado**: `FixtureRunner` compara 16 archivos de prueba contra bitácoras esperadas, verifica la copia byte a byte y el invariante "concatenar lexemas = texto fuente".

Guion sugerido para la exposición (20 minutos)

Minuto	Tema	Qué decir / mostrar
0–2	Introducción	Objetivo: analizador léxico de PHP en Java puro. Entrada: un `.php` junto al jar. Salida: `<nombre>-errors.log` + resumen en consola. Restricciones: sin librerías, sin regex, Java 8.
2–4	Arquitectura	Paquetes `app`, `io`, `lexer`, `lexer.pattern`, `validation`, `errors`, `symbols`, `report`. Las 4 etapas y en qué paquete vive cada una. `Compilador.main` solo delega.
4–6	Entrada	`PhpFileLocator`: carpeta del jar vía `toURI()`, extensión exacta `.php`. `SourceFileReader`: bytes crudos, UTF-8 estricto o ISO-8859-1, BOM, líneas con su terminador.
6–10	Etapas 1–3	`CharacterStream` (peek/advance/mark/reset), `Lexer` con modos INLINE/PHP, `PatternRegistry` con orden de prueba, un autómata con estados (ej. `StringPattern` o `NumberPattern`), coincidencia más larga en operadores, `TokenClassifier` y `ReservedWords`.
10–14	Etapa 4	`ValidationPipeline` en orden fijo. Etiquetas (100–116), identificadores con `AssignmentTargetExtractor` (200–205), comentarios (300–304), lexemas (400–402). Bandera `reported`.
14–16	Salida	`ErrorLogWriter`: `NNNN + espacio + contenido + terminador`, separador si la última línea no tiene salto, línea en blanco, errores ordenados. `ConsoleReporter` y tabla de símbolos.
16–18	Demostración	`java -jar compilador.jar` en `dist` → "No lexical errors found."; luego un archivo con errores (identifiers.php) y `--tokens`.

Minuto	Tema	Qué decir / mostrar
18–20	Pruebas y cierre	FixtureRunner (16 fixtures), invariantes, decisiones de diseño (mayúsculas, `?>` en comentarios, recuperación de cadenas). Abrir a preguntas.

> **Cómo decirlo en la defensa**
>
> Empieza siempre por el **qué** (entrada → salida), luego el **cómo** (4 etapas), y deja los detalles para cuando el profesor pregunte. Si te preguntan algo concreto, ubícalo primero en su etapa: "eso pasa en la Etapa 3, en la clase `NumberPattern`…".

Comandos que debes saber de memoria

```
ant clean jar                                  (o NetBeans: Clean and Build)
cd dist
java -jar compilador.jar                       (analiza todos los .php junto al jar)
java -jar compilador.jar codigo.php            (un archivo concreto)
java -jar compilador.jar --tokens              (también imprime la tabla de tokens)
type codigo-errors.log                         (ver la bitácora en CMD)

javac -encoding UTF-8 -d build\fixtures -cp build\classes test\compilador\FixtureRunner.java
java -cp build\classes;build\fixtures compilador.FixtureRunner test-fixtures
```

2. Conceptos teóricos que el profesor espera escuchar

¿Qué es el análisis léxico?

Es la **primera fase de un compilador**. Recibe el programa como una secuencia de caracteres y la agrupa en unidades con significado llamadas **tokens**. Las fases siguientes (análisis sintáctico, semántico, generación de código) ya no trabajan con caracteres sino con tokens. El analizador léxico también descarta o marca lo que no aporta significado (espacios, comentarios) y detecta **errores léxicos**: caracteres desconocidos, cadenas sin cerrar, identificadores mal formados, etc.

Token, lexema y patrón

Concepto	Definición	Ejemplo en este proyecto
Lexema	La secuencia exacta de caracteres del fuente.	`$suma_notas`, `=`, `"Carlos"`, `<?php`
Token	El lexema + su tipo (categoría) + su posición.	`Token(VARIABLE, "$suma_notas", línea 19, col 5)`
Patrón	La regla que describe qué lexemas pertenecen a un tipo. Normalmente se escribe como expresión regular.	`VariablePattern` ≡ regex `\$[A-Za-z0-9_]*`
Tipo / categoría	Clasificación del token.	`TokenType.ASSIGN` → `TokenCategory.OPERATOR`

Expresiones regulares y autómatas finitos

Toda expresión regular puede reconocerse con un **autómata finito determinista (AFD)**: un conjunto de estados, un estado inicial, estados de aceptación y una función de transición que, dado el estado actual y el siguiente carácter, indica el próximo estado. Herramientas como JFlex generan ese autómata automáticamente. **En este proyecto los autómatas se escribieron a mano**: cada clase `*Pattern` tiene un `enum State` (en los patrones complejos) y un ciclo que aplica transiciones carácter por carácter. Así se hace visible la etapa de *determinación de patrones (lógica equivalente)*.

Coincidencia más larga (maximal munch)

Cuando varios lexemas pueden empezar en la misma posición, el analizador elige **el más largo**. Ejemplo: ante `===` no se producen tres `=`, sino un solo operador `===`. `OperatorPattern` ordena su tabla del operador más largo al más corto y toma el primero que coincide. Los demás patrones también consumen "todo lo que pueden" (`$suma_notas` completo, `12.5e3` completo).

Prioridad entre patrones

Si dos patrones pueden empezar en el mismo carácter, gana el que se prueba primero. Por eso el orden del `PatternRegistry` importa: por ejemplo, el comentario `//` debe probarse **antes** que el operador `/`, y la etiqueta `?>` antes que el operador `?`.

Modos (estados léxicos)

PHP mezcla código con texto (HTML). El analizador tiene dos **modos**: `INLINE` (fuera de PHP, solo se busca `<?`) y `PHP` (se prueban todos los patrones). `OPEN_TAG` cambia a `PHP` y `CLOSE_TAG` regresa a `INLINE`. Es el equivalente a los *start conditions* de Lex/JFlex.

Recuperación de errores

Un buen analizador léxico **no se detiene en el primer error**: lo reporta y sigue, para mostrar todos los problemas. Ejemplos: un carácter desconocido se convierte en un token `UNKNOWN` de un carácter; una cadena sin cerrar termina al final de su línea (así no "se traga" el `?>`); si no hay `<?`, se analiza todo en modo PHP para que corran las demás validaciones.

Tabla de símbolos

Estructura que registra los identificadores encontrados. Aquí `SymbolTable` guarda cada variable con su **texto exacto** (distingue mayúsculas: `$total` ≠ `$Total`), la primera línea donde aparece y el número de apariciones.

Las cuatro etapas léxicas que pide el curso

Etapa	Qué hace	Dónde está
1. Obtención (adquisición) de tokens	Recorre el texto carácter a carácter, corta lexemas y crea tokens con posición.	`lexer`: `CharacterStream`, `Lexer`, `LexerMode`, `LexemeInvariant`
2. Clasificación de tokens	Asigna el tipo final: palabra → `KEYWORD`/`IDENTIFIER`; operador → `ASSIGN`, `SEMICOLON`… y su categoría.	`lexer`: `Token`, `TokenType`, `TokenCategory`, `TokenClassifier`, `ReservedWords`
3. Determinación de patrones	Autómatas escritos a mano que reconocen cada clase de lexema, cada uno con su regex equivalente.	`lexer.pattern`: `LexemePattern`, `PatternRegistry`, 11 patrones, `AsciiChars`, `PatternMatch`
4. Validación de lexemas	Revisa reglas sobre la lista de tokens y produce errores específicos.	`validation`: 5 validadores + contexto, extractor, utilidades

3. El problema: requisitos y restricciones

Lo que pide el enunciado

1. **Lectura del archivo:** localizar un `.php` en la misma carpeta del `.jar`, sin nombre ni ruta fijos, aceptando solo la extensión exacta `.php`, y leerlo **sin alterar su formato**.
2. **Bitácora de errores:** generar `<nombre>-errors.log` con cada línea numerada a 4 dígitos y, al final, la lista de errores específicos (uno por problema, varios por línea si aplica).
3. **Validar** etiquetas de apertura `<?php` y cierre `?>`, identificadores a la izquierda de `=`, y comentarios de línea (`//`, `#`).
4. Entregar documentación de usuario en español: `INFORME.txt`.

Restricciones duras (PRD §2)

#	Restricción	Cómo se cumple
C1	Proyecto NetBeans *Java with Ant*.	`build.xml` estándar; única personalización: target `-post-jar` que copia `codigo.php` e `INFORME.txt` a `dist/`.
C2	Sin librerías externas.	Solo imports `java.*`. Ni JFlex, ni ANTLR, ni JUnit.
C3	Sin `java.util.regex` en léxico y validadores.	Autómatas manuales; el Javadoc de cada patrón da su regex equivalente.
C4	Compatible con Java 8.	Sin `var`, records, `List.of`, `String.repeat`, `isBlank`, switch expressions…
C5	Se ejecuta con `java -jar` sin argumentos.	`PhpFileLocator` busca solo los `.php` junto al jar.
C6	Orientado a objetos y modular.	Un paquete por etapa, una responsabilidad por clase.

Archivo de entrada de referencia (codigo.php)

El archivo entregado por el profesor tiene 53 líneas, saltos `\n` y **no tiene salto de línea después de** `?>`. Con él, el programa debe producir “No lexical errors found.”. Salida real de consola:

```
PHP Lexical Analyzer
Ignored   : INFORME.txt (extension '.txt' is not '.php')
Ignored   : README.TXT (extension '.TXT' is not '.php')

File      : C:\dev\compilador\dist\codigo.php
Lines     : 53   Tokens: 274   Errors: 0
Log       : C:\dev\compilador\dist\codigo-errors.log
Identifiers (case-sensitive symbol table):
  $nombre_estudiante    first line 0005   occurrences 1
  $cantidad_notas       first line 0006   occurrences 4
  $notas                first line 0009   occurrences 3
  $suma_notas           first line 0016   occurrences 4
  $i                    first line 0018   occurrences 5
  $promedio             first line 0022   occurrences 2
  $contador_revision    first line 0035   occurrences 5
  $mensaje_final        first line 0043   occurrences 2
  $total                first line 0047   occurrences 2
  $Total                first line 0048   occurrences 2
  $resultado            first line 0050   occurrences 1
```

Salida real de `java -jar compilador.jar` en `dist/` (código de salida 0).

Final de la bitácora `codigo-errors.log`:

```
0051 echo "\nResultado final: $resultado\n";
0052
0053 ?>
```

```
No lexical errors found.
```

> **Para recordar**
>
> `$nombre_estudiante` aparece **1 vez** aunque se usa dentro de `"Estudiante: $nombre_estudiante\n"`: la interpolación dentro de cadenas está fuera de alcance; la cadena completa es **un solo token** `STRING_LITERAL`, así que las variables que contiene no se cuentan.

4. Arquitectura general

Paquetes y clases

Paquete	Clases	Responsabilidad
`compilador`	`Compilador`	Punto de entrada. Solo `System.exit(new Application().run(args))`.
`compilador.app`	`Application`, `FileAnalyzer`, `AnalysisResult`, `CliOptions`, `ExitCode`	Orquestación del flujo completo, opciones de consola y códigos de salida.
`compilador.io`	`PhpFileLocator`, `SourceFileReader`, `SourceFile`, `SourceLine`, `ErrorLogWriter`, `LogFormat`	Encontrar y leer el `.php` sin alterarlo; escribir la bitácora.
`compilador.lexer`	`CharacterStream`, `Lexer`, `LexerMode`, `LexemeInvariant` (Etapa 1) · `Token`, `TokenType`, `TokenCategory`, `TokenClassifier`, `ReservedWords` (Etapa 2)	Obtención y clasificación de tokens.
`compilador.lexer.pattern`	`LexemePattern`, `PatternMatch`, `PatternRegistry`, `AsciiChars` y 11 patrones	Etapa 3: autómatas que reconocen cada tipo de lexema.
`compilador.validation`	`Validator`, `ValidationPipeline`, `ValidationContext`, `LineText`, `AssignmentTarget`, `AssignmentTargetExtractor`, 5 validadores	Etapa 4: reglas léxicas y errores.
`compilador.errors`	`ErrorCode`, `LexicalError`, `ErrorCollector`	Modelo de errores: catálogo, error individual, colector.
`compilador.symbols`	`SymbolTable`, `SymbolEntry`	Tabla de símbolos sensible a mayúsculas.
`compilador.report`	`ConsoleReporter`	Única clase (junto con `Application`) que imprime en consola.
`test/compilador`	`FixtureRunner`	Pruebas de regresión sin JUnit.

Diagrama del flujo de datos

```
java -jar compilador.jar [archivo.php] [-t|--tokens]
          |
Compilador.main ──> Application.run(args)
          |
          |-- CliOptions.parse(args)                opciones de consola
          |-- PhpFileLocator.locate(arg)            ¿qué .php analizar?
          |
          |   por cada archivo (orden alfabético):
          |-- FileAnalyzer.read      -> SourceFileReader.read   => SourceFile (líneas + terminadores)
          |-- FileAnalyzer.analyze:
          |      Lexer.tokenize(rawText)                         [Etapas 1, 2 y 3]
          |         CharacterStream  --peek/advance-->  PatternRegistry --> XxxPattern.match
          |                                             TokenClassifier.classify  => List<Token>
          |      SymbolTable.build(tokens)
          |      ValidationPipeline.validate(source, tokens)     [Etapa 4]
          |         Opening -> Closing -> Identifier -> Comment -> Lexeme  => ErrorCollector
          |      => AnalysisResult (source, tokens, symbols, errores ordenados)
          |-- FileAnalyzer.writeLog  -> ErrorLogWriter.write   => <nombre>-errors.log
          |-- ConsoleReporter: resumen, símbolos, tokens (opcional)
          v
System.exit(0 ó 1)
```

Recorrido completo de los datos a través de las clases.

Principios de diseño aplicados

- **Responsabilidad única:** cada clase hace una cosa (el localizador no lee, el lector no escribe, cada validador revisa un grupo de reglas).
- **Patrón Strategy:** `LexemePattern` y `Validator` son interfaces; el `Lexer` y el `ValidationPipeline` recorren listas de implementaciones sin conocer los detalles de cada una.
- **Objetos de valor inmutables:** `Token` (salvo la bandera `reported`), `SourceLine`, `SourceFile`, `LexicalError`, `PatternMatch`, `AnalysisResult` tienen campos `final` y listas no modificables.
- **Constantes en lugar de "números mágicos":** plantillas de error, separador de bitácora, palabras reservadas, operadores, extensiones.
- **Separación de E/S y lógica:** `FileAnalyzer` no imprime nada (por eso `FixtureRunner` puede reutilizarlo); solo `ConsoleReporter` y `Application` escriben en consola.

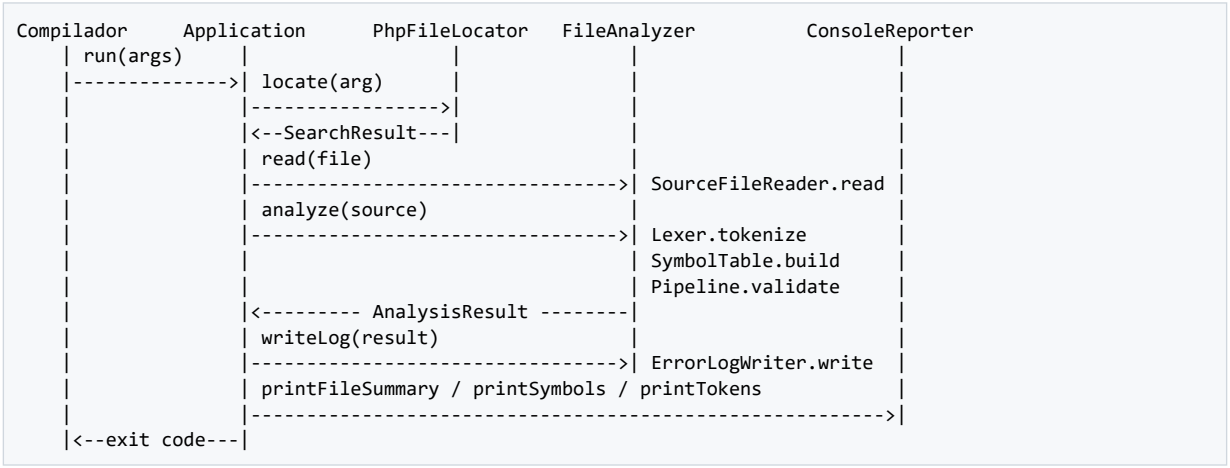
5. Recorrido secuencial completo del programa

Este es el capítulo más importante para la defensa: explica, **en orden**, qué ocurre desde que se escribe `java -jar compilador.jar` hasta que el proceso termina. Memoriza los 12 pasos.

- **Paso 1 — Arranque.** La JVM lee `Main-Class: compilador.Compilador` del manifiesto del jar y ejecuta `main`. `main` crea `Application` y llama `run(args)`; el entero que devuelve se pasa a `System.exit`.
- **Paso 2 — Encabezado y opciones.** `Application.run` imprime "PHP Lexical Analyzer" y llama `CliOptions.parse(args)`: reconoce `-t`/`--tokens` y como máximo un nombre de archivo. Opción desconocida o dos archivos → `IllegalArgumentException` → mensaje y código 1.
- **Paso 3 — Directorio base.** `PhpFileLocator.searchDirectories` obtiene la carpeta del jar con `getProtectionDomain().getCodeSource().getLocation().toURI()`. Si esa ubicación es un directorio (ejecución desde NetBeans: `build/classes`) usa `user.dir`. Si `user.dir` es distinto, se agrega como segundo lugar de búsqueda.
- **Paso 4 — Selección de archivos.** Sin argumento: lista los archivos regulares que terminan **exactamente** en `.php` (sensible a mayúsculas), en orden alfabético; los casi-candidatos (`x.PHP`, `x.php.txt`, `x.txt`) se informan como "Ignored" con su motivo. Con argumento: si no termina en `.php` → mensaje de extensión incorrecta y código 1; si no existe → "not found" y código 1.
- **Paso 5 — Lectura.** Para cada archivo, `SourceFileReader.read` lee **todos los bytes**, detecta el BOM (`EF BB BF`), decodifica como UTF-8 estricto (o ISO-8859-1 si falla) y divide en `SourceLine` conservando cada terminador (`\r\n`, `\n`, `\r` o vacío). Resultado: `SourceFile`.
- **Paso 6 — Tokenización (Etapas 1–3).** `Lexer.tokenize(rawText)`: decide el modo inicial (INLINE si existe `<?`, PHP si no) y, mientras queden caracteres, llama `nextToken`. En modo PHP, `PatternRegistry` devuelve el primer patrón cuyo `canStart` es verdadero y su `match` consume el lexema.
- **Paso 7 — Clasificación (Etapa 2).** Cada `PatternMatch` pasa por `TokenClassifier.classify`: palabras → `KEYWORD`/`IDENTIFIER`; operadores → `ASSIGN`, `COMPARISON_OPERATOR`, `SEMICOLON`, etc. Se crea el `Token` con línea, columna y línea final; `OPEN_TAG`/`CLOSE_TAG` cambian el modo.
- **Paso 8 — Tabla de símbolos.** `SymbolTable.build(tokens)` registra cada token `VARIABLE` con nombre bien formado, con su texto exacto, primera línea y apariciones.
- **Paso 9 — Validación (Etapa 4).** `ValidationPipeline.validate` crea un `ValidationContext` y un `ErrorCollector` y ejecuta, en orden: `OpeningTagValidator`, `ClosingTagValidator`, `IdentifierValidator`, `CommentValidator`, `LexemeValidator`.
- **Paso 10 — Ordenamiento.** `ErrorCollector.getSorted()` ordena por línea, columna y código (`LexicalError.compareTo`). Todo queda en un `AnalysisResult` inmutable.
- **Paso 11 — Bitácora.** `ErrorLogWriter.write` arma el texto: copia numerada (`NNNN` + espacio + contenido + terminador), separador extra si la última línea no tenía salto, una línea vacía y los errores (o "No lexical errors found."). Escribe con el **mismo charset** del fuente, junto al `.php`.
- **Paso 12 — Consola y salida.** `ConsoleReporter` imprime ruta, líneas, tokens, errores, ruta de la bitácora y la tabla de símbolos; con `--tokens` también la tabla de tokens y si se cumple el invariante. Código de salida 0 si se analizó todo; 1 si algún archivo no se pudo leer o escribir.

> **Cómo decirlo en la defensa**
> "El `main` no tiene lógica: delega en `Application`. `Application` coordina: localiza, lee, tokeniza, construye la tabla de símbolos, valida, escribe la bitácora y reporta. Cada paso es una clase distinta, y las cuatro etapas léxicas viven en paquetes separados."

Diagrama de secuencia simplificado



6. Paquete app: punto de entrada y orquestación

Compilador (clase principal)

Es la clase indicada en `main.class` de `nbproject/project.properties`. Por regla del proyecto **no contiene lógica**: crea la aplicación, la ejecuta y termina el proceso con el código devuelto.

```
package compilador;

// Proyecto desarrollado con apoyo de IA generativa (Claude Code, Anthropic), revisado por el
   estudiante; ver sección 12 de INFORME.txt.
/** Punto de entrada del analizador léxico de PHP; no pertenece a ninguna etapa léxica, solo delega en
   Application. */
public class Compilador {

    /** Crea la aplicación, la ejecuta con los argumentos de consola y termina con su código de salida.
       */
    public static void main(String[] args) {
        System.exit(new Application().run(args));
    }
}
```

Archivo completo: `Compilador.java`

> **Cómo decirlo en la defensa**
> "Se separó `main` de la lógica para que la aplicación sea testeable: `run` devuelve un entero en vez de llamar `System.exit` por dentro, así se puede invocar desde otro código sin matar la JVM."

ExitCode

Clase de constantes: `SUCCESS = 0` (se analizó al menos un archivo; los errores léxicos **no** cuentan como fallo) y `FAILURE = 1` (no se encontró archivo, extensión incorrecta o error de E/S). Constructor privado para que no se instancie.

CliOptions

Interpreta los argumentos de consola. Es inmutable y se crea solo mediante el método de fábrica `parse` (constructor privado).

```
/** Interpreta los argumentos; lanza IllegalArgumentException ante una opción desconocida o más de un
   archivo. */
public static CliOptions parse(String[] args) {
    String file = null;
    boolean tokens = false;
    for (String arg : args) {
        if (SHORT_TOKENS_FLAG.equals(arg) || LONG_TOKENS_FLAG.equals(arg)) {
            tokens = true;
        } else if (arg.startsWith(OPTION_PREFIX)) {
            throw new IllegalArgumentException("unknown option '" + arg + "'; the only option is "
                    + SHORT_TOKENS_FLAG + " / " + LONG_TOKENS_FLAG + ".");
        } else if (file != null) {
            throw new IllegalArgumentException("only one file can be given, but both '" + file
                    + "' and '" + arg + "' were.");
        } else {
            file = arg;
        }
    }
    return new CliOptions(file, tokens);
}
```

Fuente: `app/CliOptions.java`

- `-t` o `--tokens` activa la impresión de la tabla de tokens.

- Cualquier otro argumento que empiece con `-` es una opción desconocida → excepción.
- El primer argumento restante es el archivo; un segundo archivo → excepción.
- `getFileArgument()` devuelve el archivo o `null` (búsqueda automática); `isShowTokens()` la bandera.

Application

Orquesta todo. Tiene tres colaboradores como campos: `PhpFileLocator`, `FileAnalyzer` y `ConsoleReporter(System.out)`.

run(String[] args)

```
/** Ejecuta el analizador con los argumentos de consola y devuelve un código de ExitCode. */
public int run(String[] args) {
    reporter.printHeader();
    CliOptions options;
    try {
        options = CliOptions.parse(args);
    } catch (IllegalArgumentException e) {
        reporter.printUsageError(e.getMessage());
        return ExitCode.FAILURE;
    }
    SearchResult search;
    try {
        search = locator.locate(options.getFileArgument());
    } catch (IOException e) {
        reporter.printSearchError(e.getMessage());
        return ExitCode.FAILURE;
    }
    reporter.printIgnored(search.getIgnored());
    if (!reportSearchProblem(search)) {
        return ExitCode.FAILURE;
    }
    return analyzeAll(search.getFiles(), options.isShowTokens());
}
```

Fuente: `app/Application.java`

Secuencia: encabezado → opciones → búsqueda → listar ignorados → verificar problema de búsqueda → analizar todos. Cada fallo temprano devuelve `ExitCode.FAILURE`.

reportSearchProblem(SearchResult)

```
/** Devuelve true si se encontraron archivos; si no, imprime el motivo y devuelve false. */
private boolean reportSearchProblem(SearchResult search) {
    switch (search.getStatus()) {
        case FOUND:
            return true;
        case WRONG_EXTENSION:
            reporter.printWrongExtension(search.getArgument());
            return false;
        case FILE_NOT_FOUND:
            reporter.printFileNotFound(search.getArgument(), search.getSearchedDirectories());
            return false;
        default:
            reporter.printNoPhpFound(search.getSearchedDirectories());
            return false;
    }
}
```

Fuente: `app/Application.java`

Usa un `switch` clásico (compatible con Java 8) sobre el `Status` del enum de `PhpFileLocator`. Devuelve `true` solo si el estado es `FOUND`.

analyzeAll y analyze

```
/** Analiza cada archivo sin detenerse si uno falla; devuelve FAILURE si alguno no se pudo procesar. */
private int analyzeAll(List<Path> files, boolean showTokens) {
    int exitCode = ExitCode.SUCCESS;
    for (Path file : files) {
        if (!analyze(file, showTokens)) {
            exitCode = ExitCode.FAILURE;
        }
    }
    return exitCode;
}
```

Fuente: `app/Application.java`

```
/** Ejecuta el flujo completo sobre un archivo; devuelve false si no se pudo leer o escribir su
   bitácora. */
private boolean analyze(Path file, boolean showTokens) {
    SourceFile source;
    try {
        source = analyzer.read(file);
    } catch (IOException e) {
        reporter.printReadError(file, describe(e));
        return false;
    }
    AnalysisResult result = analyzer.analyze(source);
    Path logPath;
    try {
        logPath = analyzer.writeLog(result);
    } catch (IOException e) {
        reporter.printLogWriteError(ErrorLogWriter.logPathFor(file), describe(e));
        return false;
    }
    reporter.printFileSummary(source, result.getTokens().size(), result.getErrors().size(),
            logPath);
    reporter.printSymbols(result.getSymbols());
    // La tabla de tokens solo se imprime si el usuario pasó -t/--tokens.
    if (showTokens) {
        reporter.printTokens(result.getTokens(),
                LexemeInvariant.holds(result.getTokens(), source.getRawText()));
    }
    return true;
}
```

Fuente: `app/Application.java`

`analyzeAll` **no se detiene** si un archivo falla: sigue con los demás y al final devuelve 1 si alguno falló. `analyze` ejecuta el flujo de un archivo: leer, analizar, escribir la bitácora y reportar. Los errores de lectura o escritura se capturan y se informan con un motivo legible.

describe(IOException)

```
/** Arma un motivo legible del error de E/S, ya que las excepciones del sistema de archivos solo traen
   la ruta. */
private static String describe(IOException e) {
    String detail = e instanceof FileSystemException
            ? ((FileSystemException) e).getReason() : e.getMessage();
    String type = e.getClass().getSimpleName();
    return detail == null ? type : type + " (" + detail + ")";
}
```

Fuente: `app/Application.java`

Las excepciones del sistema de archivos (`AccessDeniedException`, `NoSuchFileException`) traen solo la ruta en `getMessage()`; por eso se usa `getReason()` cuando es una `FileSystemException`, y se antepone el nombre de la clase de la excepción.

FileAnalyzer

Encadena las etapas para **un** archivo sin imprimir nada. Al no tener salida de consola, lo usa tanto `Application` como `FixtureRunner` (el mismo flujo que el jar).

```
package compilador.app;

/** Flujo de análisis de un archivo (leer, tokenizar, tabla de símbolos, validar, bitácora) sin
   imprimir nada; encadena las etapas 1 a 4. */
public final class FileAnalyzer {

    // Componentes de cada paso del flujo: lectura, etapas léxicas, validación y escritura de la
       bitácora.
    private final SourceFileReader reader = new SourceFileReader();
    private final Lexer lexer = new Lexer();
    private final ValidationPipeline validation = new ValidationPipeline();
    private final ErrorLogWriter logWriter = new ErrorLogWriter();

    /** Lee un archivo sin alterar su formato (PRD §3.2); lanza IOException si no se puede leer. */
    public SourceFile read(Path file) throws IOException {
        return reader.read(file);
    }

    /** Ejecuta las cuatro etapas léxicas sobre un archivo ya leído y devuelve el resultado. */
    public AnalysisResult analyze(SourceFile source) {
        List<Token> tokens = lexer.tokenize(source.getRawText());
        SymbolTable symbols = SymbolTable.build(tokens);
        return new AnalysisResult(source, tokens, symbols,
                validation.validate(source, tokens).getSorted());
    }

    /** Escribe <nombre>-errors.log junto al archivo fuente (PRD §4) y devuelve su ruta. */
    public Path writeLog(AnalysisResult result) throws IOException {
        return logWriter.write(result.getSource(), result.getErrors());
    }
}
```

Archivo completo: `app/FileAnalyzer.java`

> **Para recordar**
> El método `analyze` es el corazón del proyecto en tres líneas: `tokenize` (Etapas 1–3), `SymbolTable.build` y `validate(...).getSorted()` (Etapa 4).

AnalysisResult

Objeto de valor inmutable con el resultado de analizar un archivo: `SourceFile`, lista de tokens, tabla de símbolos y errores **ya ordenados**. El constructor copia las listas y las envuelve con `Collections.unmodifiableList`, así nadie puede modificarlas después.

```
/** Guarda copias no modificables de los tokens y de los errores (ya ordenados por línea, columna y
   código). */
public AnalysisResult(SourceFile source, List<Token> tokens, SymbolTable symbols,
        List<LexicalError> errors) {
    this.source = source;
    this.tokens = Collections.unmodifiableList(new ArrayList<Token>(tokens));
    this.symbols = symbols;
    this.errors = Collections.unmodifiableList(new ArrayList<LexicalError>(errors));
}
```

Fuente: `app/AnalysisResult.java`

7. Paquete io: localizar, leer y escribir

PhpFileLocator — encontrar el archivo sin ruta fija

Implementa PRD §3.1. Tiene la constante pública `PHP_EXTENSION = ".php"`, un enum `Status` (`FOUND`, `NO_PHP_FOUND`, `WRONG_EXTENSION`, `FILE_NOT_FOUND`) y dos clases anidadas inmutables: `IgnoredFile` (ruta + motivo) y `SearchResult` (estado, archivos, ignorados, directorios revisados, argumento).

locate(String fileArgument)

```
/** Busca los archivos a analizar: el indicado en la consola o todos los .php del directorio base. */
public SearchResult locate(String fileArgument) throws IOException {
    List<Path> directories = searchDirectories();
    if (fileArgument != null) {
        return locateArgument(fileArgument, directories);
    }
    List<IgnoredFile> ignored = new ArrayList<IgnoredFile>();
    // Se detiene en el primer directorio que contenga archivos .php.
    for (Path directory : directories) {
        List<Path> files = listPhpFiles(directory, ignored);
        if (!files.isEmpty()) {
            return new SearchResult(Status.FOUND, files, ignored, directories, null);
        }
    }
    return new SearchResult(Status.NO_PHP_FOUND, Collections.<Path>emptyList(), ignored,
            directories, null);
}
```

Fuente: `io/PhpFileLocator.java`

Si hay argumento, delega en `locateArgument`. Si no, recorre los directorios de búsqueda y se **detiene en el primero** que contenga algún `.php`.

searchDirectories y resolveBaseDirectory

```
/** Devuelve primero el directorio base y luego user.dir si es distinto. */
private static List<Path> searchDirectories() {
    List<Path> directories = new ArrayList<Path>();
    Path userDir = Paths.get(System.getProperty(USER_DIR_PROPERTY)).toAbsolutePath()
            .normalize();
    Path baseDir = resolveBaseDirectory(userDir);
    directories.add(baseDir);
    if (!baseDir.equals(userDir)) {
        directories.add(userDir);
    }
    return directories;
}
```

Fuente: `io/PhpFileLocator.java`

```
/** Obtiene la carpeta del jar con toURI() (soporta espacios y tildes); usa userDir si se ejecuta desde
   un directorio de clases. */
private static Path resolveBaseDirectory(Path userDir) {
    try {
        CodeSource codeSource = PhpFileLocator.class.getProtectionDomain().getCodeSource();
        URL location = codeSource == null ? null : codeSource.getLocation();
        if (location == null) {
            return userDir;
        }
        Path codePath = Paths.get(location.toURI()).toAbsolutePath().normalize();
        if (Files.isDirectory(codePath) || codePath.getParent() == null) {
            return userDir;
        }
        return codePath.getParent();
    } catch (URISyntaxException | SecurityException | IllegalArgumentException e) {
        return userDir;
    }
}
```

Fuente: `io/PhpFileLocator.java`

- `getCodeSource().getLocation()` da la URL del jar (p. ej. `file:/C:/curso/dist/compilador.jar`).
- Se convierte con `toURI()` y `Paths.get(URI)` para que funcionen **espacios y tildes** en la ruta (una URL los codifica como `%20`; convertir la URL directamente a texto rompería la ruta).
- Si la ubicación es un directorio (ejecución desde NetBeans con `build/classes`) se usa `user.dir`.
- Ante cualquier excepción (`URISyntaxException`, `SecurityException`…) también se usa `user.dir`.

locateArgument y resolveArgument

```
/** Valida la extensión del argumento de consola y verifica que el archivo exista. */
private static SearchResult locateArgument(String argument, List<Path> directories) {
    List<IgnoredFile> none = Collections.emptyList();
    if (!argument.endsWith(PHP_EXTENSION)) {
        return new SearchResult(Status.WRONG_EXTENSION, Collections.<Path>emptyList(),
                none, directories, argument);
    }
    Path file = resolveArgument(argument, directories);
    if (file == null) {
        return new SearchResult(Status.FILE_NOT_FOUND, Collections.<Path>emptyList(),
                none, directories, argument);
    }
    return new SearchResult(Status.FOUND, Collections.singletonList(file), none,
            directories, argument);
}
```

Fuente: `io/PhpFileLocator.java`

```
/** Devuelve el archivo existente que nombra el argumento o null; una ruta relativa se prueba en cada
   directorio en orden. */
private static Path resolveArgument(String argument, List<Path> directories) {
    try {
        Path given = Paths.get(argument);
        if (given.isAbsolute()) {
            return Files.isRegularFile(given) ? given.normalize() : null;
        }
        for (Path directory : directories) {
            Path candidate = directory.resolve(given).normalize();
            if (Files.isRegularFile(candidate)) {
                return candidate;
            }
        }
        return null;
    } catch (InvalidPathException e) {
        return null;
    }
}
```

Fuente: `io/PhpFileLocator.java`

La extensión se valida **antes** de buscar el archivo. Una ruta relativa se prueba primero contra el directorio base y luego contra `user.dir`.

listPhpFiles, isCandidate y rejectionReason

```
/** Lista en orden alfabético los archivos aceptados de un directorio y agrega los rechazados a
   ignored. */
private static List<Path> listPhpFiles(Path directory, List<IgnoredFile> ignored)
        throws IOException {
    List<Path> accepted = new ArrayList<Path>();
    try (DirectoryStream<Path> entries = Files.newDirectoryStream(directory)) {
        for (Path entry : entries) {
            // Solo se consideran archivos regulares (no carpetas).
            if (!Files.isRegularFile(entry)) {
                continue;
            }
            String reason = rejectionReason(entry.getFileName().toString());
            if (reason == null) {
                accepted.add(entry);
            } else if (isCandidate(entry.getFileName().toString())) {
                ignored.add(new IgnoredFile(entry, reason));
            }
        }
    }
    Collections.sort(accepted);
    Collections.sort(ignored);
    return accepted;
}
```

Fuente: `io/PhpFileLocator.java`

```
/** Devuelve null si el nombre se acepta o, si no, el motivo del rechazo. */
static String rejectionReason(String name) {
    if (name.endsWith(PHP_EXTENSION)) {
        return name.length() > PHP_EXTENSION.length()
                ? null : "the name has nothing before '" + PHP_EXTENSION + "'";
    }
    int dot = name.lastIndexOf(EXTENSION_DOT);
    if (dot < 0) {
        return "it has no extension; '" + PHP_EXTENSION + "' is required";
    }
    String extension = name.substring(dot);
    // Caso x.PHP: la extensión coincide solo si se ignoran las mayúsculas.
    if (extension.toLowerCase(Locale.ROOT).equals(PHP_EXTENSION)) {
        return "extension '" + extension + "' is not exactly '" + PHP_EXTENSION
                + "'; the check is case-sensitive";
    }
    // Caso x.php.txt: .php no es la extensión final.
    if (name.contains(PHP_EXTENSION + EXTENSION_DOT)) {
        return "'" + PHP_EXTENSION + "' is not the final extension; the name ends in '"
                + extension + "'";
    }
    return "extension '" + extension + "' is not '" + PHP_EXTENSION + "'";
}
```

Fuente: `io/PhpFileLocator.java`

`rejectionReason` devuelve `null` si el nombre se acepta, o un motivo específico: sin extensión, `.PHP` (difiere solo en mayúsculas), `.php.txt` (`.php` no es la extensión final), u otra extensión. Un archivo llamado solo `.php` se rechaza (no tiene nombre). `isCandidate` evita listar como “ignorados” el propio `.jar` y las bitácoras `-errors.log`.

> **Para recordar**
>
> `endsWith(".php")` es sensible a mayúsculas: `x.PHP`, `x.php.txt` y `x.phtml` se rechazan, tal como exige el PRD.

SourceLine y SourceFile — el archivo en memoria

`SourceLine` es inmutable: `number` (desde 1), `content` (sin terminador) y `terminator` (`"\r\n"`, `"\n"`, `"\r"` o `""`). `hasTerminator()` indica si termina con salto. Nada se recorta ni se normaliza.

`SourceFile` guarda la ruta, el `Charset` usado, si tenía BOM, el texto decodificado sin BOM (`rawText`) y la lista de líneas. Métodos: `getLineCount`, `getLastLine` y:

```
/** Devuelve el separador para líneas añadidas a la bitácora: el primer terminador del archivo o el del
   sistema (PRD §4). */
public String lineSeparatorForAppending() {
    for (SourceLine line : lines) {
        if (line.hasTerminator()) {
            return line.getTerminator();
        }
    }
    return System.lineSeparator();
}
```

Fuente: `io/SourceFile.java`

Ese separador se usa para las líneas que **agrega** la bitácora (la línea en blanco y los errores): el primer terminador que aparezca en el archivo, o el del sistema si no tiene ninguno. Así un archivo CRLF produce una bitácora CRLF consistente.

SourceFileReader — leer sin alterar el formato

read(Path)

```
/** Lee y decodifica el archivo: UTF-8 estricto sin BOM o, si falla, ISO-8859-1. */
public SourceFile read(Path path) throws IOException {
    byte[] bytes = Files.readAllBytes(path);
    boolean hasBom = startsWithBom(bytes);
    int offset = hasBom ? UTF8_BOM.length : 0;
    String text = decodeStrictUtf8(bytes, offset);
    Charset charset = StandardCharsets.UTF_8;
    // Si no es UTF-8 válido, los primeros bytes tampoco eran un BOM: se usa ISO-8859-1.
    if (text == null) {
        hasBom = false;
        text = new String(bytes, StandardCharsets.ISO_8859_1);
        charset = StandardCharsets.ISO_8859_1;
    }
    return new SourceFile(path, charset, hasBom, text, splitLines(text));
}
```

Fuente: `io/SourceFileReader.java`

1. `Files.readAllBytes`: se leen los bytes crudos (no `readAllLines`, que perdería los terminadores).
2. `startsWithBom`: compara los 3 primeros bytes con `EF BB BF`.
3. `decodeStrictUtf8`: decodifica desde después del BOM con un `CharsetDecoder` en modo `REPORT`; si hay bytes inválidos lanza `CharacterCodingException` y el método devuelve `null`.
4. Si falla, se decodifica todo como ISO-8859-1 (que acepta cualquier byte) y se asume que no había BOM.
5. Se divide en líneas con `splitLines`.

```
/** Decodifica como UTF-8 estricto desde el desplazamiento dado; devuelve null si los bytes no son
   UTF-8 válido. */
private static String decodeStrictUtf8(byte[] bytes, int offset) {
    CharsetDecoder decoder = StandardCharsets.UTF_8.newDecoder()
            .onMalformedInput(CodingErrorAction.REPORT)
            .onUnmappableCharacter(CodingErrorAction.REPORT);
    try {
        return decoder.decode(ByteBuffer.wrap(bytes, offset, bytes.length - offset))
                .toString();
    } catch (CharacterCodingException e) {
        return null;
    }
}
```

Fuente: `io/SourceFileReader.java`

splitLines y terminatorAt

```
/** Divide el texto en líneas con su terminador original; texto vacío da una línea vacía y el último
   terminador no crea línea extra. */
static List<SourceLine> splitLines(String text) {
    List<SourceLine> lines = new ArrayList<SourceLine>();
    int start = 0;
    int i = 0;
    while (i < text.length()) {
        char c = text.charAt(i);
        // Avanza hasta encontrar un \r o \n.
        if (c != CR && c != LF) {
            i++;
            continue;
        }
        String terminator = terminatorAt(text, i);
        lines.add(new SourceLine(lines.size() + 1, text.substring(start, i), terminator));
        i += terminator.length();
        start = i;
    }
    // Agrega la última línea sin terminador, o una línea vacía si el texto estaba vacío.
    if (start < text.length() || lines.isEmpty()) {
        lines.add(new SourceLine(lines.size() + 1, text.substring(start), NO_TERMINATOR));
    }
    return lines;
}
```

Fuente: `io/SourceFileReader.java`

```
/** Devuelve el terminador que empieza en la posición dada: "\r\n", "\r" o "\n". */
private static String terminatorAt(String text, int index) {
    if (text.charAt(index) == CR) {
        boolean followedByLf = index + 1 < text.length() && text.charAt(index + 1) == LF;
        return followedByLf ? CRLF : String.valueOf(CR);
    }
    return String.valueOf(LF);
}
```

Fuente: `io/SourceFileReader.java`

- Recorre el texto; al encontrar `\r` o `\n` crea una línea con el contenido desde `start` y el terminador.
- `\r` seguido de `\n` es un único terminador `\r\n`.
- Al final solo agrega una línea si quedó texto **o** si no había ninguna (archivo vacío → una línea vacía).
- Por eso `"?>\n"` produce **una** línea `?>` con terminador `\n`, sin línea fantasma.

Texto de entrada	Líneas resultantes
`""` (archivo vacío)	1: `""` + `""`
`"<?php\n?>"`	1: `<?php` + `\n` · 2: `?>` + `""`
`"?>\n"`	1: `?>` + `\n` (no hay línea 2)
`"a\r\nb\rc"`	1: `a` + `\r\n` · 2: `b` + `\r` · 3: `c` + `""`

LogFormat — formato de la bitácora

```
package compilador.io;

/** Constantes y reglas de formato de la bitácora de errores (PRD §4); formato de salida de los
   resultados de la Etapa 4. */
public final class LogFormat {

    /** Sufijo que reemplaza ".php" en el nombre de la bitácora: codigo-errors.log. */
    public static final String LOG_FILE_SUFFIX = "-errors.log";

    /** Texto entre el número de línea y el contenido copiado; cambiarlo a "" pega el número al texto.
       */
    public static final String NUMBER_SEPARATOR = " ";

    /** Cantidad mínima de dígitos del número de línea; los números más largos nunca se truncan. */
    public static final int LINE_NUMBER_WIDTH = 4;

    /** Última línea de la bitácora cuando el análisis no encontró problemas. */
    public static final String NO_ERRORS_MESSAGE = "No lexical errors found.";

    // Formato que rellena con ceros a la izquierda: "%04d".
    private static final String LINE_NUMBER_FORMAT = "%0" + LINE_NUMBER_WIDTH + "d";

    // Constructor privado: clase de utilidades que no se instancia.
    private LogFormat() {
    }

    /** Rellena el número de línea con ceros a 4 dígitos: 5 -> "0005", 12345 -> "12345". */
    public static String formatLineNumber(int lineNumber) {
        return String.format(LINE_NUMBER_FORMAT, lineNumber);
    }
}
```

Archivo completo: `io/LogFormat.java`

`"%0" + 4 + "d"` es `"%04d"`: rellena con ceros a 4 dígitos; si el número tiene más dígitos (línea 12345) simplemente se imprime completo, el ancho crece solo.

ErrorLogWriter — escribir <nombre>-errors.log

```
/** Escribe la bitácora con el charset del fuente, reemplaza la anterior y devuelve su ruta. */
public Path write(SourceFile source, List<LexicalError> errors) throws IOException {
    Path logPath = logPathFor(source.getPath());
    String content = render(source, errors);
    Files.write(logPath, content.getBytes(source.getCharset()));
    return logPath;
}
```

Fuente: `io/ErrorLogWriter.java`

```
/** Calcula la ruta de la bitácora en la carpeta del fuente: codigo.php -> codigo-errors.log. */
public static Path logPathFor(Path sourcePath) {
    String name = sourcePath.getFileName().toString();
    String baseName = name.endsWith(PhpFileLocator.PHP_EXTENSION)
            ? name.substring(0, name.length() - PhpFileLocator.PHP_EXTENSION.length())
            : name;
    return sourcePath.resolveSibling(baseName + LogFormat.LOG_FILE_SUFFIX);
}
```

Fuente: `io/ErrorLogWriter.java`

```
/** Construye el texto completo de la bitácora: copia numerada y sección de errores. */
static String render(SourceFile source, List<LexicalError> errors) {
    StringBuilder log = new StringBuilder();
    appendNumberedCopy(log, source);
    appendErrorSection(log, source, errors);
    return log.toString();
}
```

Fuente: `io/ErrorLogWriter.java`

```
/** Agrega cada línea como NNNN + separador + contenido + terminador original. */
private static void appendNumberedCopy(StringBuilder log, SourceFile source) {
    for (SourceLine line : source.getLines()) {
        log.append(LogFormat.formatLineNumber(line.getNumber()))
                .append(LogFormat.NUMBER_SEPARATOR)
                .append(line.getContent())
                .append(line.getTerminator());
    }
}
```

Fuente: `io/ErrorLogWriter.java`

```
/** Agrega una línea en blanco y luego los errores, o el mensaje de que no hay errores. */
private static void appendErrorSection(StringBuilder log, SourceFile source,
        List<LexicalError> errors) {
    String separator = source.lineSeparatorForAppending();
    // Evita que la sección de errores quede pegada a la última línea copiada.
    if (!source.getLastLine().hasTerminator()) {
        log.append(separator);
    }
    log.append(separator);
    if (errors.isEmpty()) {
        log.append(LogFormat.NO_ERRORS_MESSAGE).append(separator);
        return;
    }
    for (LexicalError error : errors) {
        log.append(error.toLogLine()).append(separator);
    }
}
```

Fuente: `io/ErrorLogWriter.java`

- `logPathFor`: quita el `.php` final y agrega `-errors.log`; `resolveSibling` lo coloca en la **misma carpeta** del fuente.
- `appendNumberedCopy`: `0001` + `" "` + contenido + **terminador original**. Por eso la copia es idéntica byte a byte (quitando los 5 primeros caracteres).
- `appendErrorSection`: si la última línea no tiene terminador (caso normal de un archivo correcto que termina en `?>`), primero agrega un salto para que el primer error no quede pegado a `?>`; luego una línea en blanco; luego los errores o el mensaje de “sin errores”.
- `Files.write(..., content.getBytes(source.getCharset()))` sobrescribe la bitácora anterior y usa el mismo charset del fuente; el BOM **no** se reproduce.

8. Etapa 1 — Obtención (adquisición) de tokens

La Etapa 1 recorre el texto **carácter por carácter**, decide dónde empieza y termina cada lexema y crea los tokens con su posición. Clases: `LexerMode`, `CharacterStream`, `Lexer` y `LexemeInvariant`.

LexerMode

Enum con dos valores: `INLINE` (fuera de PHP: solo se reconoce `<?`, todo lo demás es `INLINE_TEXT`) y `PHP` (se prueban todos los patrones).

CharacterStream — el lector de caracteres

Envuelve el texto (sin BOM) y lleva `position` (índice desde 0), `line` y `column` (desde 1), `lastConsumedLine` (línea del último carácter consumido; sirve para `endLine` de tokens multilínea) y el `mode` actual. La constante `END = -1` indica "más allá del final".

peek, lookingAt, isAtEnd

```
/** Devuelve el carácter a "offset" posiciones adelante (0 = siguiente) sin consumirlo, o END. */
public int peek(int offset) {
    int index = position + offset;
    return index < text.length() ? text.charAt(index) : END;
}
```

Fuente: `lexer/CharacterStream.java`

```
/** Indica si el texto en la posición actual comienza con la cadena esperada. */
public boolean lookingAt(String expected) {
    return text.startsWith(expected, position);
}
```

Fuente: `lexer/CharacterStream.java`

`peek(k)` mira `k` caracteres adelante **sin consumir** (lookahead). Devuelve `int` para poder devolver `END`. `lookingAt` usa `String.startsWith(prefijo, posición)` para comparar texto fijo como `<?`, `?>`, `//`, `/*`.

advance — consumir y actualizar línea/columna

```
/** Consume un carácter y actualiza línea y columna; \r\n, \r y \n cuentan como un solo salto. */
public char advance() {
    if (isAtEnd()) {
        throw new IllegalStateException("advance() past the end of the text");
    }
    char c = text.charAt(position++);
    lastConsumedLine = line;
    // Un \r seguido de \n no cambia de línea: el salto se cuenta al consumir el \n.
    boolean crBeforeLf = c == CR && peek() == LF;
    if ((c == LF || c == CR) && !crBeforeLf) {
        line++;
        column = 1;
    } else {
        column++;
    }
    return c;
}
```

Fuente: `lexer/CharacterStream.java`

Regla clave: `\r\n` cuenta como **un solo** salto. Al consumir el `\r` de un par `\r\n` solo avanza la columna; el salto de línea se cuenta al consumir el `\n`. Un `\r` o `\n` aislado incrementa la línea y reinicia la columna a 1.

mark, reset y textSince — retroceso

```
/** Toma una instantánea de la posición actual para poder volver con reset(). */
public Mark mark() {
    return new Mark(position, line, column, lastConsumedLine);
}
```

Fuente: `lexer/CharacterStream.java`

```
/** Regresa a una posición marcada previamente (retroceso). */
public void reset(Mark mark) {
    position = mark.position;
    line = mark.line;
    column = mark.column;
    lastConsumedLine = mark.lastConsumedLine;
}
```

Fuente: `lexer/CharacterStream.java`

```
/** Devuelve el texto consumido desde la marca indicada. */
public String textSince(Mark start) {
    return text.substring(start.position, position);
}
```

Fuente: `lexer/CharacterStream.java`

`Mark` es una clase anidada inmutable (posición, línea, columna, última línea consumida) con constructor privado: solo `CharacterStream` puede crear marcas. Se usa para: (1) obtener el lexema consumido con `textSince`, y (2) **retroceder** con `reset` cuando un patrón necesita deshacer lo leído (`OpenTagPattern` con espacios sin letras, `StringPattern` en la recuperación de cadenas sin cerrar).

Lexer — el bucle principal

tokenize(String text)

```
/** Tokeniza el texto (sin BOM) y devuelve todos los tokens en orden, incluidos espacios y saltos de
   línea. */
public List<Token> tokenize(String text) {
    // Si no hay ningún "<?", todo se analiza en modo PHP para que las demás validaciones corran (error
       103).
    LexerMode initialMode = text.contains(OPEN_TAG_START) ? LexerMode.INLINE : LexerMode.PHP;
    CharacterStream stream = new CharacterStream(text, initialMode);
    List<Token> tokens = new ArrayList<Token>();
    while (!stream.isAtEnd()) {
        tokens.add(nextToken(stream));
    }
    return tokens;
}
```

Fuente: `lexer/Lexer.java`

Si el texto **no contiene** `<?`, empieza directamente en modo PHP (recuperación del error 103: así se siguen validando identificadores y comentarios). Luego produce tokens hasta consumir todo el texto. Nunca se descarta ningún carácter: espacios, saltos y comentarios también son tokens.

nextToken

```
/** Reconoce el siguiente lexema según el modo, lo clasifica, actualiza el modo y crea el token. */
private Token nextToken(CharacterStream stream) {
    Mark start = stream.mark();
    PatternMatch match = stream.getMode() == LexerMode.INLINE
            ? matchInline(stream) : matchPhp(stream);
    TokenType type = classifier.classify(match.getType(), match.getLexeme());
    updateMode(stream, type);
    return new Token(type, match.getLexeme(), start.getLine(),
            start.getColumn(), stream.getLastConsumedLine());
}
```

Fuente: `lexer/Lexer.java`

1. Marca la posición inicial (para la línea y columna del token).
2. Según el modo, llama `matchInline` o `matchPhp` → obtiene un `PatternMatch` (tipo provisional + lexema).
3. **Etapa 2:** `classifier.classify` asigna el tipo definitivo.
4. `updateMode`: `OPEN_TAG` → modo PHP; `CLOSE_TAG` → modo INLINE.
5. Crea el `Token` con tipo, lexema, línea/columna de inicio y `lastConsumedLine` como línea final.

matchInline, matchPhp, matchUnknown, updateMode

```
/** En modo INLINE reconoce OPEN_TAG o consume texto hasta el siguiente "<?" como INLINE_TEXT. */
private PatternMatch matchInline(CharacterStream stream) {
    LexemePattern openTag = registry.getOpenTagPattern();
    if (openTag.canStart(stream)) {
        return openTag.match(stream);
    }
    Mark start = stream.mark();
    while (!stream.isAtEnd() && !openTag.canStart(stream)) {
        stream.advance();
    }
    return new PatternMatch(TokenType.INLINE_TEXT, stream.textSince(start));
}
```

Fuente: `lexer/Lexer.java`

```
/** En modo PHP prueba los patrones en orden; si ninguno aplica, produce un token UNKNOWN. */
private PatternMatch matchPhp(CharacterStream stream) {
    LexemePattern pattern = registry.findPhpPattern(stream);
    if (pattern != null) {
        return pattern.match(stream);
    }
    return matchUnknown(stream);
}
```

Fuente: `lexer/Lexer.java`

```
/** Consume un punto de código completo (ambas mitades de un par sustituto) como UNKNOWN. */
private static PatternMatch matchUnknown(CharacterStream stream) {
    Mark start = stream.mark();
    char first = stream.advance();
    if (Character.isHighSurrogate(first) && !stream.isAtEnd()
            && Character.isLowSurrogate((char) stream.peek())) {
        stream.advance();
    }
    return new PatternMatch(TokenType.UNKNOWN, stream.textSince(start));
}
```

Fuente: `lexer/Lexer.java`

```
/** OPEN_TAG cambia a modo PHP y CLOSE_TAG regresa a modo INLINE. */
private static void updateMode(CharacterStream stream, TokenType type) {
    if (type == TokenType.OPEN_TAG) {
        stream.setMode(LexerMode.PHP);
    } else if (type == TokenType.CLOSE_TAG) {
        stream.setMode(LexerMode.INLINE);
    }
}
```

Fuente: `lexer/Lexer.java`

`matchUnknown` es la red de seguridad: si ningún patrón acepta el carácter (p. ej. `€`, `ñ` suelta, `\u00a0`), produce un token `UNKNOWN` de **un punto de código** (si es un emoji formado por un par sustituto UTF-16, consume las dos mitades). Así el análisis nunca se bloquea y el error 401 se reporta después.

LexemeInvariant — autocomprobación

```
/** Devuelve la posición (desde 0) del primer carácter distinto, o -1 si la concatenación es igual al
   texto. */
public static int firstMismatch(List<Token> tokens, String rawText) {
    String joined = concatenate(tokens);
    int common = Math.min(joined.length(), rawText.length());
    for (int i = 0; i < common; i++) {
        if (joined.charAt(i) != rawText.charAt(i)) {
            return i;
        }
    }
    // Si el prefijo común coincide, solo difieren si las longitudes son distintas.
    return joined.length() == rawText.length() ? -1 : common;
}
```

Fuente: `lexer/LexemeInvariant.java`

Verifica que **concatenar todos los lexemas reproduce exactamente el texto**. Si se cumple, el lexer no perdió, duplicó ni alteró ningún carácter. `firstMismatch` devuelve la primera posición distinta o -1. Lo usan `--tokens` (imprime “Lexeme concatenation equals the source text: yes”) y `FixtureRunner`.

9. Etapa 2 — Clasificación de tokens

La Etapa 3 determina **qué patrón** reconoció el lexema y le da un tipo provisional; la Etapa 2 decide el **tipo final** y su **categoría**. Clases: `TokenType`, `TokenCategory`, `Token`, `TokenClassifier`, `ReservedWords`.

TokenType y TokenCategory

Categoría (TokenCategory)	Tipos (TokenType)
TAG	OPEN_TAG, CLOSE_TAG
INLINE_TEXT	INLINE_TEXT
IDENTIFIER	VARIABLE, IDENTIFIER
KEYWORD	KEYWORD
LITERAL	INTEGER_LITERAL, FLOAT_LITERAL, STRING_LITERAL
COMMENT	INLINE_COMMENT, BLOCK_COMMENT
OPERATOR	ASSIGN, COMPARISON_OPERATOR, ARITHMETIC_OPERATOR, LOGICAL_OPERATOR, CONCAT_OPERATOR, OTHER_OPERATOR
DELIMITER	SEMICOLON, COMMA, LPAREN, RPAREN, LBRACE, RBRACE, LBRACKET, RBRACKET
LAYOUT	WHITESPACE, NEWLINE
ERROR	UNKNOWN, MALFORMED_NUMBER, UNTERMINATED_STRING, UNTERMINATED_COMMENT

Cada constante de `TokenType` recibe su categoría en el constructor del enum, así la relación tipo → categoría está en un único lugar:

```
// Categoría general a la que pertenece el tipo.
private final TokenCategory category;

// Asocia cada tipo con su categoría.
TokenType(TokenCategory category) {
    this.category = category;
}

/** Devuelve la categoría general del tipo de token. */
public TokenCategory getCategory() {
    return category;
}
```

Fuente: `lexer/TokenType.java` *(líneas 52–64)*

Token

Campos `final`: `type`, `lexeme`, `line`, `column`, `endLine`. El **único** campo mutable es `reported`, que los validadores usan para no reportar dos veces el mismo token.

```
/** Crea el token; el lexema no puede ser vacío y endLine difiere de line solo si abarca varias líneas.
   */
public Token(TokenType type, String lexeme, int line, int column, int endLine) {
    if (type == null || lexeme == null || lexeme.isEmpty()) {
        throw new IllegalArgumentException("type and a non-empty lexeme are required");
    }
    this.type = type;
    this.lexeme = lexeme;
    this.line = line;
    this.column = column;
    this.endLine = endLine;
}
```

Fuente: `lexer/Token.java`

```
/** Marca el token como reportado para que los validadores siguientes lo omitan. */
public void markReported() {
```

```
    reported = true;
}
```

Fuente: `lexer/Token.java`

Otros métodos: `getCategory()` (delegado en el tipo), `is(TokenType)` para comparaciones legibles y `toString()` con formato `TIPO@línea:columna[lexema]` para depuración.

ReservedWords

```
package compilador.lexer;

/** Las 20 palabras reservadas (PRD §5.3) y sus dos reglas de mayúsculas; Etapa 2: clasificación de
   tokens. */
public final class ReservedWords {

    /** La verificación de $nombre distingue mayúsculas: $if es error pero $If no. */
    public static final boolean IDENTIFIER_CHECK_CASE_SENSITIVE = true;

    /** La clasificación de palabras sueltas como KEYWORD ignora mayúsculas, como PHP (ECHO es palabra
       clave). */
    public static final boolean KEYWORD_CLASSIFICATION_CASE_SENSITIVE = false;

    /** Palabras reservadas en minúscula; "Include" del enunciado se trata como "include". */
    public static final Set<String> WORDS = Collections.unmodifiableSet(new HashSet<String>(
            Arrays.asList("class", "function", "if", "while", "echo", "else", "continue",
                    "for", "return", "public", "null", "false", "true", "trim", "foreach",
                    "private", "include", "print", "const", "do")));

    // Constructor privado: clase de utilidades que no se instancia.
    private ReservedWords() {
    }

    /** Indica si el nombre de variable (sin $) es reservado y no puede usarse como identificador. */
    public static boolean isReservedIdentifierName(String name) {
        return contains(name, IDENTIFIER_CHECK_CASE_SENSITIVE);
    }

    /** Indica si una palabra suelta se clasifica como KEYWORD. */
    public static boolean isKeyword(String word) {
        return contains(word, KEYWORD_CLASSIFICATION_CASE_SENSITIVE);
    }

    /** Busca la palabra en la lista, pasándola a minúscula si la comparación no distingue mayúsculas.
       */
    private static boolean contains(String word, boolean caseSensitive) {
        return WORDS.contains(caseSensitive ? word : word.toLowerCase(Locale.ROOT));
    }
}
```

Archivo completo: `lexer/ReservedWords.java`

- Las **20 palabras** del enunciado, en minúscula (“Include” se trata como `include`).
- `IDENTIFIER_CHECK_CASE_SENSITIVE = true`: `$if` es error 204, `$If` no.
- `KEYWORD_CLASSIFICATION_CASE_SENSITIVE = false`: la palabra suelta `ECHO` se clasifica como `KEYWORD`, igual que en PHP real.
- Las dos políticas son constantes documentadas: cambiarlas no requiere tocar la lógica.

TokenClassifier

```
/** Palabras sueltas pasan a KEYWORD si son reservadas; operadores se buscan en la tabla; los demás
   tipos quedan igual. */
public TokenType classify(TokenType determined, String lexeme) {
    if (determined == TokenType.IDENTIFIER) {
        return ReservedWords.isKeyword(lexeme) ? TokenType.KEYWORD : TokenType.IDENTIFIER;
    }
    // Operadores sin tipo propio (+=, .=, ->, =>, ::, ??...) quedan como OTHER_OPERATOR.
    if (determined == TokenType.OTHER_OPERATOR) {
        TokenType type = OPERATOR_TYPES.get(lexeme);
        return type == null ? TokenType.OTHER_OPERATOR : type;
    }
    return determined;
}
```

Fuente: `lexer/TokenClassifier.java`

```
/** Construye la tabla de operadores: "=" es ASSIGN y los de comparación, aritmética, lógica,
   concatenación y delimitadores tienen su tipo. */
private static Map<String, TokenType> buildOperatorTypes() {
    Map<String, TokenType> types = new HashMap<String, TokenType>();
    types.put("=", TokenType.ASSIGN);
    put(types, TokenType.COMPARISON_OPERATOR,
            "==", "===", "!=", "!==", "<>", "<", ">", "<=", ">=", "<=>");
    put(types, TokenType.ARITHMETIC_OPERATOR, "+", "-", "*", "/", "%", "**", "++", "--");
    put(types, TokenType.LOGICAL_OPERATOR, "&&", "||", "!");
    types.put(".", TokenType.CONCAT_OPERATOR);
    types.put(";", TokenType.SEMICOLON);
    types.put(",", TokenType.COMMA);
    types.put("(", TokenType.LPAREN);
    types.put(")", TokenType.RPAREN);
    types.put("{", TokenType.LBRACE);
    types.put("}", TokenType.RBRACE);
    types.put("[", TokenType.LBRACKET);
    types.put("]", TokenType.RBRACKET);
    return Collections.unmodifiableMap(types);
}
```

Fuente: `lexer/TokenClassifier.java`

Dos decisiones: (1) un `IDENTIFIER` provisional (palabra sin `$`) pasa a `KEYWORD` si es reservada; (2) un `OTHER_OPERATOR` provisional se busca en la tabla: `=` → `ASSIGN`, `==` → `COMPARISON_OPERATOR`, `;` → `SEMICOLON`, etc. Los operadores sin entrada (`+=`, `.=`, `->`, `=>`, `::`, `??`…) se quedan como `OTHER_OPERATOR`. El resto de tipos pasa sin cambios.

> **Para recordar**
> Distinguir `ASSIGN` (`=`) de `==`, `===`, `+=`, `.=`, `=>` es lo que permite que la validación de identificadores se active **solo** con el operador de asignación simple, como pide el PRD §5.3.

10. Etapa 3 — Determinación de patrones (autómatas)

Cada clase de esta etapa reconoce un tipo de lexema con un **autómata escrito a mano**. Ninguna usa `java.util.regex`; en su Javadoc está la expresión regular **equivalente**.

Contrato: LexemePattern y PatternMatch

```
package compilador.lexer.pattern;

/** Reconocedor escrito a mano para un tipo de lexema, equivalente a una expresión regular sin usar
   regex; Etapa 3: determinación de patrones. */
public interface LexemePattern {

    /** Mira hacia adelante sin consumir e indica si este patrón reconoce un lexema que empieza aquí. */
    boolean canStart(CharacterStream stream);

    /** Consume el lexema más largo que reconoce el patrón (al menos un carácter); solo se llama si
       canStart devolvió true. */
    PatternMatch match(CharacterStream stream);
}
```

Archivo completo: `lexer/pattern/LexemePattern.java`

`canStart` mira hacia adelante **sin consumir** (decide si el patrón aplica); `match` consume el lexema más largo posible y devuelve un `PatternMatch` (tipo + lexema no vacío). `PatternMatch` valida en su constructor que el lexema no sea vacío: esto garantiza que el lexer **siempre avanza** y no entra en un ciclo infinito.

PatternRegistry — el orden de prueba

```
/** Registra los patrones PHP en el orden en que se prueban; el primero que acepta la posición gana. */
public PatternRegistry() {
    List<LexemePattern> patterns = new ArrayList<LexemePattern>();
    patterns.add(new NewlinePattern());
    patterns.add(new WhitespacePattern());
    patterns.add(new CloseTagPattern());
    patterns.add(new InlineCommentPattern());
    patterns.add(new BlockCommentPattern());
    patterns.add(new StringPattern());
    patterns.add(new VariablePattern());
    patterns.add(new NumberPattern());
    patterns.add(new WordPattern());
    patterns.add(new OperatorPattern());
    this.phpPatterns = Collections.unmodifiableList(patterns);
}
```

Fuente: `lexer/pattern/PatternRegistry.java`

```
/** Devuelve el primer patrón PHP que puede empezar en la posición actual, o null (el léxico emitirá
   UNKNOWN). */
public LexemePattern findPhpPattern(CharacterStream stream) {
    for (LexemePattern pattern : phpPatterns) {
        if (pattern.canStart(stream)) {
            return pattern;
        }
    }
    return null;
}
```

Fuente: `lexer/pattern/PatternRegistry.java`

#	Patrón	Por qué en esta posición
1	`NewlinePattern`	Los saltos son lo más frecuente; aislados para llevar las líneas.

#	Patrón	Por qué en esta posición
2	`WhitespacePattern`	Espacios y tabs.
3	`CloseTagPattern`	Antes que operadores: si no, `?>` serían `?` y `>`.
4	`InlineCommentPattern`	Antes que operadores: si no, `//` serían dos `/`.
5	`BlockCommentPattern`	Antes que operadores: si no, `/*` serían `/` y `*`.
6	`StringPattern`	La cadena se consume completa; lo que contiene (`//`, `#`, `$x`) nunca se analiza.
7	`VariablePattern`	Empieza con `$`.
8	`NumberPattern`	Antes que operadores: `.5` es número, no `.` + `5`.
9	`WordPattern`	Letras o `_`.
10	`OperatorPattern`	Último: coincidencia más larga sobre la tabla de operadores.
—	`UNKNOWN` (en `Lexer`)	Si ninguno aplica.

El patrón `OpenTagPattern` no está en la lista PHP: es el único que se usa en modo INLINE y se obtiene con `getOpenTagPattern()`.

AsciiChars — clases de caracteres

Funciones `isLetter` (`[A-Za-z]`), `isDigit` (`[0-9]`), `isNameStart` (`[A-Za-z_]`), `isNameChar` (`[A-Za-z0-9_]`) e `isLineBreak` (`\r` o `\n`). Se comparan rangos ASCII a propósito: `ñ` **no es letra** (identificadores solo ASCII, decisión del PRD). No se usa `Character.isLetter`, que sí aceptaría `ñ`.

Patrones simples

NewlinePattern — regex `\r\n|\r|\n`

```
/** Consume \r\n como un solo salto, o bien un \r o \n aislado. */
@Override
public PatternMatch match(CharacterStream stream) {
    Mark start = stream.mark();
    char first = stream.advance();
    if (first == CR && stream.peek() == LF) {
        stream.advance();
    }
    return new PatternMatch(TokenType.NEWLINE, stream.textSince(start));
}
```

Fuente: `lexer/pattern/NewlinePattern.java`

WhitespacePattern — regex `[ \t]+`

Consume espacios y tabulaciones consecutivos. Los saltos de línea **no** son whitespace aquí (son `NEWLINE` aparte), lo que permite a los validadores razonar por líneas.

CloseTagPattern — regex `\?>`

`canStart` = `lookingAt("?>")`. El operador nullsafe `?->` no empieza con `?>`, así que nunca se confunde: lo reconoce `OperatorPattern`.

InlineCommentPattern — regex `(//|#)[^\r\n]*`

```
/** Consume hasta antes del terminador de línea o el final del texto. */
@Override
public PatternMatch match(CharacterStream stream) {
    Mark start = stream.mark();
    while (!stream.isAtEnd() && !AsciiChars.isLineBreak(stream.peek())) {
        stream.advance();
    }
    return new PatternMatch(TokenType.INLINE_COMMENT, stream.textSince(start));
}
```

Fuente: `lexer/pattern/InlineCommentPattern.java`

Consume hasta **antes** del salto de línea (el terminador queda como `NEWLINE`). Todo lo que hay dentro, incluido `?>`, forma parte del lexema del comentario y **no produce tokens** (desviación deliberada de PHP real, documentada).

WordPattern — regex `[A-Za-z_][A-Za-z0-9_]*`

```
/** Consume el primer carácter y luego letras, dígitos y guiones bajos. */
@Override
public PatternMatch match(CharacterStream stream) {
    Mark start = stream.mark();
    stream.advance();
    while (AsciiChars.isNameChar(stream.peek())) {
        stream.advance();
    }
    return new PatternMatch(TokenType.IDENTIFIER, stream.textSince(start));
}
```

Fuente: `lexer/pattern/WordPattern.java`

Devuelve el tipo provisional `IDENTIFIER`; la Etapa 2 decide si es `KEYWORD`.

OpenTagPattern — regex `<\?(=|[ \t]*[pP]([hH][pP]?)?)?`

```
/** Consume "<?" y luego "=" o espacios con letras de "php"; toda variante es OPEN_TAG y el validador
   reporta las incorrectas. */
@Override
public PatternMatch match(CharacterStream stream) {
    Mark start = stream.mark();
    stream.advance();
    stream.advance();
    if (stream.peek() == SHORT_ECHO) {
        stream.advance();
    } else {
        matchSpacedLetters(stream);
    }
    return new PatternMatch(TokenType.OPEN_TAG, stream.textSince(start));
}
```

Fuente: `lexer/pattern/OpenTagPattern.java`

```
/** Consume [ \t]* seguido de un prefijo de "php" sin importar mayúsculas; si no hay letras, no consume
   nada. */
private static void matchSpacedLetters(CharacterStream stream) {
    Mark afterTagStart = stream.mark();
    while (stream.peek() == ' ' || stream.peek() == '\t') {
        stream.advance();
    }
    int letters = 0;
    while (letters < TAG_LETTERS.length() && matchesIgnoreCase(stream.peek(),
            TAG_LETTERS.charAt(letters))) {
        stream.advance();
        letters++;
    }
    // Los espacios solo forman parte de la etiqueta si les siguen letras.
    if (letters == 0) {
        stream.reset(afterTagStart);
    }
}
```

Fuente: `lexer/pattern/OpenTagPattern.java`

Acepta variantes mal formadas (`<?PHP`, `<? php`, `<?`, `<?=`, `<?ph`) como `OPEN_TAG` para poder **reportarlas** después con el error 102 en vez de fallar. Si después de `<?` hay espacios pero no letras de “php”, se retrocede con `reset` para que los espacios no formen parte de la etiqueta.

Autómatas con enum State

VariablePattern — regex `\$[A-Za-z0-9_]*`

```
/** Consume el $ y luego letras, dígitos y guiones bajos ASCII; un carácter no ASCII como ñ termina el
   token. */
@Override
public PatternMatch match(CharacterStream stream) {
    Mark start = stream.mark();
    State state = State.DOLLAR;
    while (state != State.DONE) {
        if (state == State.DOLLAR) {
            stream.advance();
            state = State.NAME;
        } else if (AsciiChars.isNameChar(stream.peek())) {
            stream.advance();
        } else {
            state = State.DONE;
        }
    }
    return new PatternMatch(TokenType.VARIABLE, stream.textSince(start));
}
```

Fuente: `lexer/pattern/VariablePattern.java`

Estado	Entrada	Siguiente estado
`DOLLAR` (inicial)	`$`	`NAME`
`NAME`	`[A-Za-z0-9_]`	`NAME` (consume)
`NAME`	otro / fin	`DONE` (acepta)

Es **permisivo a propósito**: `$1variable` y un `$` solo también son `VARIABLE`. La decisión de si el nombre es válido le corresponde a la Etapa 4 (errores 200/201), que da mensajes específicos. Un carácter no ASCII (`ñ`) termina el token.

StringPattern — regex `q([^q\\]|\\[\s\S])*q` (q = la comilla)

```
/** Ejecuta el autómata; si no cierra, retrocede y corta la cadena al final de su línea (recuperación
   del error 400). */
@Override
public PatternMatch match(CharacterStream stream) {
    Mark start = stream.mark();
    char quote = (char) stream.peek();
    State state = State.OPEN;
    while (state != State.CLOSED && state != State.UNTERMINATED) {
        state = step(state, stream, quote);
    }
    if (state == State.CLOSED) {
        return new PatternMatch(TokenType.STRING_LITERAL, stream.textSince(start));
    }
    // Recuperación: la cadena sin cerrar no se traga un ?> posterior.
    stream.reset(start);
    consumeRestOfLine(stream);
    return new PatternMatch(TokenType.UNTERMINATED_STRING, stream.textSince(start));
}
```

Fuente: `lexer/pattern/StringPattern.java`

```
/** Realiza una transición consumiendo como máximo un carácter. */
private static State step(State state, CharacterStream stream, char quote) {
    if (stream.isAtEnd()) {
        return State.UNTERMINATED;
    }
    char c = stream.advance();
    switch (state) {
        // La comilla de apertura o el carácter escapado llevan al cuerpo.
        case OPEN:
        case ESCAPE:
            return State.BODY;
```

```
        default:
            if (c == quote) {
                return State.CLOSED;
            }
            return c == BACKSLASH ? State.ESCAPE : State.BODY;
    }
}
```

Fuente: `lexer/pattern/StringPattern.java`

Estado	Carácter leído	Siguiente
`OPEN`	la comilla de apertura	`BODY`
`BODY`	la misma comilla	`CLOSED` (acepta: `STRING_LITERAL`)
`BODY`	`\`	`ESCAPE`
`BODY`	otro	`BODY`
`ESCAPE`	cualquiera (escapado)	`BODY`
cualquiera	fin del texto	`UNTERMINATED`

Acepta comillas `'`, `"` y `` ` `. Gracias al estado ESCAPE, "dice \"hola\"" es una sola cadena. **Recuperación (error 400):** si llega al final sin cerrar, reset(start) vuelve al inicio y consumeRestOfLine corta la cadena al final de su línea. Así una comilla olvidada no se "traga" el resto del archivo ni el `?>`.

BlockCommentPattern — regex `/\*([^*]|\*+[^*/])*\*+/`

```
/** Realiza una transición consumiendo como máximo un carácter. */
private static State step(State state, CharacterStream stream) {
    // Los dos primeros estados consumen la barra y el asterisco de apertura.
    if (state == State.SLASH || state == State.OPEN_STAR) {
        stream.advance();
        return state == State.SLASH ? State.OPEN_STAR : State.BODY;
    }
    if (stream.isAtEnd()) {
        return State.UNTERMINATED;
    }
    char c = stream.advance();
    // En BODY un asterisco puede iniciar el cierre.
    if (state == State.BODY) {
        return c == STAR ? State.STAR : State.BODY;
    }
    // En STAR una barra cierra el comentario; otro asterisco se queda en STAR.
    if (c == SLASH) {
        return State.CLOSED;
    }
    return c == STAR ? State.STAR : State.BODY;
}
```

Fuente: `lexer/pattern/BlockCommentPattern.java`

Estado	Carácter	Siguiente
`SLASH`	`/`	`OPEN_STAR`
`OPEN_STAR`	`*`	`BODY`
`BODY`	`*` / otro	`STAR` / `BODY`
`STAR`	`/` / `*` / otro	`CLOSED` / `STAR` / `BODY`
`BODY` o `STAR`	fin del texto	`UNTERMINATED` → `UNTERMINATED_COMMENT`

Un comentario de bloque puede abarcar varias líneas (el token tendrá `endLine > line`), pero luego el validador lo reporta como error 303. Si nunca se cierra se reporta 304.

NumberPattern — el autómata más grande

Regex equivalente (con D = `[0-9](_?[0-9])*`): entero `D | 0[xX]hex | 0[bB]bin | 0[oO]oct`; flotante `(D\.D? | \.D)([eE][+-]?D)? | D[eE][+-]?D`; mal formado = cualquiera de los anteriores seguido de `[A-Za-z0-9_]+`.

```
/** Ejecuta el autómata, decide si es entero o flotante y marca como mal formado si siguen letras,
   dígitos o _. */
@Override
public PatternMatch match(CharacterStream stream) {
    Mark start = stream.mark();
    State state = State.START;
    boolean isFloat = false;
    while (state != State.DONE) {
        state = step(state, stream);
        isFloat |= state == State.FRACTION_START || state == State.EXPONENT;
    }
    // Caso 12abc: todo el bloque es un solo token MALFORMED_NUMBER.
    if (AsciiChars.isNameChar(stream.peek())) {
        consumeNameChars(stream);
        return new PatternMatch(TokenType.MALFORMED_NUMBER, stream.textSince(start));
    }
    TokenType type = isFloat ? TokenType.FLOAT_LITERAL : TokenType.INTEGER_LITERAL;
    return new PatternMatch(type, stream.textSince(start));
}
```

Fuente: `lexer/pattern/NumberPattern.java`

```
/** Realiza una transición del autómata consumiendo los caracteres que reconoce. */
private static State step(State state, CharacterStream stream) {
    switch (state) {
        case START:
            return fromStart(stream);
        case INTEGER:
            return fromInteger(stream);
        case FRACTION_START:
            if (acceptDigit(stream, DECIMAL, false)) {
                return State.FRACTION;
            }
            return acceptExponent(stream) ? State.EXPONENT : State.DONE;
        case FRACTION:
            if (acceptDigit(stream, DECIMAL, true)) {
                return State.FRACTION;
            }
            return acceptExponent(stream) ? State.EXPONENT : State.DONE;
        case EXPONENT:
            return acceptDigit(stream, DECIMAL, true) ? State.EXPONENT : State.DONE;
        case HEX:
            return acceptDigit(stream, HEXADECIMAL, true) ? State.HEX : State.DONE;
        case BINARY:
            return acceptDigit(stream, BINARY_RADIX, true) ? State.BINARY : State.DONE;
        case OCTAL:
            return acceptDigit(stream, OCTAL_RADIX, true) ? State.OCTAL : State.DONE;
        default:
            return State.DONE;
    }
}
```

Fuente: `lexer/pattern/NumberPattern.java`

Estado	Qué consume	Transiciones
`START`	primer carácter	`0x`/`0b`/`0o` + dígito → `HEX`/`BINARY`/`OCTAL`; `.` → `FRACTION_START`; dígito → `INTEGER`
`INTEGER`	dígitos o `_dígito`	`.` (no `..`) → `FRACTION_START`; `e`/`E` → `EXPONENT`; otro → `DONE`
`FRACTION_START`	primer dígito decimal	dígito → `FRACTION`; exponente → `EXPONENT`; otro → `DONE`
`FRACTION`	dígitos o `_dígito`	exponente → `EXPONENT`; otro → `DONE`
`EXPONENT`	dígitos	otro → `DONE`

Estado	Qué consume	Transiciones
`HEX`/`BINARY`/`OCTAL`	dígitos de su base	otro → `DONE`

Si al terminar el autómata el siguiente carácter es letra, dígito o `_` (caso `12abc`), se consume todo el bloque como **un solo** token `MALFORMED_NUMBER` (error 402). La variable `isFloat` se activa al pasar por `FRACTION_START` o `EXPONENT`. Detalles finos: `1..2` no se toma como flotante (para no romper operadores como `...`); `1e` sin dígitos no consume la `e` (y queda como `MALFORMED_NUMBER`).

OperatorPattern — coincidencia más larga

```
/** Todos los operadores y signos de puntuación reconocidos en modo PHP. */
public static final List<String> OPERATORS = Collections.unmodifiableList(Arrays.asList(
        "<<=", ">>=", "**=", "??=", "?->", "...", "<=>", "===", "!==",
        "==", "!=", "<>", "<=", ">=", "&&", "||", "++", "--",
        "+=", "-=", "*=", "/=", ".=", "%=", "&=", "|=", "^=",
        "->", "=>", "::", "<<", ">>", "??", "**",
        "=", "+", "-", "*", "/", "%", ".", "<", ">", "!", "&", "|", "^", "~",
        "?", ":", "@", ";", ",", "(", ")", "{", "}", "[", "]", "\\"));
```

Fuente: `lexer/pattern/OperatorPattern.java`

```
/** Devuelve el operador más largo con el que comienza el texto, o null si ninguno. */
private static String longestMatch(CharacterStream stream) {
    for (String operator : LONGEST_FIRST) {
        if (stream.lookingAt(operator)) {
            return operator;
        }
    }
    return null;
}
```

Fuente: `lexer/pattern/OperatorPattern.java`

```
/** Ordena la lista de operadores por longitud descendente. */
private static List<String> sortLongestFirst(List<String> operators) {
    List<String> sorted = new ArrayList<String>(operators);
    Collections.sort(sorted, new Comparator<String>() {
        // Compara por longitud para que los más largos queden primero.
        @Override
        public int compare(String a, String b) {
            return b.length() - a.length();
        }
    });
    return Collections.unmodifiableList(sorted);
}
```

Fuente: `lexer/pattern/OperatorPattern.java`

La lista se ordena **una vez** (constante estática) de mayor a menor longitud con un `Comparator` anónimo (estilo Java 8 sin lambdas). Al recorrerla, el primer operador que coincide es el más largo: `===` antes que `==` antes que `=`. Devuelve tipo provisional `OTHER_OPERATOR`; la Etapa 2 asigna el final.

11. Etapa 4 — Validación de lexemas

La Etapa 4 recibe la lista completa de tokens (incluidos espacios y saltos) y el archivo fuente, y aplica las reglas del PRD §5. Cada validador revisa un grupo de reglas y escribe en un `ErrorCollector` compartido.

Infraestructura: Validator, ValidationContext, ValidationPipeline

```
package compilador.validation;

/** Grupo de reglas léxicas del PRD §5 (etiquetas, identificadores, comentarios, lexemas); Etapa 4:
   validación de lexemas. */
public interface Validator {

    /** Revisa las reglas del validador y registra cada violación; solo puede modificar la marca
       "reported" de los tokens. */
    void validate(ValidationContext context, ErrorCollector errors);
}
```

Archivo completo: `validation/Validator.java`

`ValidationContext` agrupa lo que un validador puede inspeccionar: `getSource()`, `getTokens()` (lista no modificable), `getLine(n)` (línea fuente desde 1), `getLineCount()` y `tokensOfType(tipo)` (filtra los tokens de un tipo en orden).

```
/** Registra los validadores: apertura, cierre, identificadores, comentarios y lexemas restantes. */
public ValidationPipeline() {
    List<Validator> ordered = new ArrayList<Validator>();
    ordered.add(new OpeningTagValidator());
    ordered.add(new ClosingTagValidator());
    ordered.add(new IdentifierValidator());
    ordered.add(new CommentValidator());
    ordered.add(new LexemeValidator());
    this.validators = Collections.unmodifiableList(ordered);
}
```

Fuente: `validation/ValidationPipeline.java`

```
/** Valida un archivo con todos los validadores y devuelve el colector con todos los errores. */
public ErrorCollector validate(SourceFile source, List<Token> tokens) {
    ValidationContext context = new ValidationContext(source, tokens);
    ErrorCollector errors = new ErrorCollector();
    for (Validator validator : validators) {
        validator.validate(context, errors);
    }
    return errors;
}
```

Fuente: `validation/ValidationPipeline.java`

> **Para recordar**
>
> **El orden importa** por la bandera `reported`: `IdentifierValidator` marca los tokens de un destino inválido y `LexemeValidator` (que corre al final) omite los marcados. Así `$ñame = 1;` produce solo el error 201 y no además un 401 por el carácter `ñ`.

LineText — utilidades de texto

```
/** Indica si el texto está vacío o solo contiene espacios y tabulaciones. */
public static boolean isBlank(String text) {
    for (int i = 0; i < text.length(); i++) {
        if (!isSpaceOrTab(text.charAt(i))) {
            return false;
        }
    }
    return true;
}
```

Fuente: `validation/LineText.java`

```
/** Recorta el texto a 40 caracteres (puntos de código) y agrega "..." si era más largo. */
public static String excerpt(String text) {
    if (text.codePointCount(0, text.length()) <= MAX_EXCERPT_LENGTH) {
        return text;
    }
    return text.substring(0, text.offsetByCodePoints(0, MAX_EXCERPT_LENGTH)) + ELLIPSIS;
}
```

Fuente: `validation/LineText.java`

- `isSpaceOrTab`, `isBlank`, `trim`: versiones propias que consideran solo espacio y tabulación (`String.isBlank` no existe en Java 8 y `trim()` de Java quita también otros controles).
- `excerpt`: recorta un texto citado a **40 puntos de código** y agrega `...` (regla del PRD §5).
- `lineOfIndex`: dentro de un token multilínea, calcula en qué línea cae un índice (se usa para la pista del `?>` dentro de un comentario).

OpeningTagValidator — códigos 100 a 106

```
/** Reporta BOM (106) y ausencia (103); el primer OPEN_TAG es "la" etiqueta y cada uno posterior es
   adicional (105). */
@Override
public void validate(ValidationContext context, ErrorCollector errors) {
    if (context.getSource().hadBom()) {
        errors.add(ErrorCode.OPEN_BOM, FIRST_LINE, WHOLE_LINE, BOM);
    }
    List<Token> openTags = context.tokensOfType(TokenType.OPEN_TAG);
    if (openTags.isEmpty()) {
        errors.add(ErrorCode.OPEN_MISSING, FIRST_LINE, WHOLE_LINE, MISSING);
        return;
    }
    Token tag = openTags.get(0);
    checkLinesBefore(context, tag, errors);
    checkSameLinePrefix(context, tag, errors);
    checkLexeme(context, tag, errors);
    // Toda etiqueta de apertura después de la primera es un error 105.
    for (int i = 1; i < openTags.size(); i++) {
        Token extra = openTags.get(i);
        errors.add(ErrorCode.OPEN_ADDITIONAL, extra.getLine(), extra.getColumn(),
                String.format(ADDITIONAL, LineText.excerpt(extra.getLexeme()),
                        extra.getColumn()));
    }
}
```

Fuente: `validation/OpeningTagValidator.java`

```
/** Reporta 101 por cada línea en blanco o 100 por cada línea con texto antes de la línea de la
   etiqueta. */
private static void checkLinesBefore(ValidationContext context, Token tag,
        ErrorCollector errors) {
    for (int number = 1; number < tag.getLine(); number++) {
        String content = context.getLine(number).getContent();
        String lineLabel = LogFormat.formatLineNumber(number);
        if (LineText.isBlank(content)) {
            errors.add(ErrorCode.OPEN_BLANK_BEFORE, number, WHOLE_LINE,
                    String.format(BLANK_LINE_BEFORE, lineLabel));
        } else {
```

```
            errors.add(ErrorCode.OPEN_TEXT_BEFORE, number, WHOLE_LINE,
                    String.format(TEXT_BEFORE_LINE,
                            LineText.excerpt(LineText.trim(content)), lineLabel));
        }
    }
}
```

Fuente: `validation/OpeningTagValidator.java`

```
/** Reporta 101 si solo hay espacios, o 100 si hay texto, antes de la etiqueta en su misma línea. */
private static void checkSameLinePrefix(ValidationContext context, Token tag,
        ErrorCollector errors) {
    String prefix = context.getLine(tag.getLine()).getContent()
            .substring(0, tag.getColumn() - 1);
    if (prefix.isEmpty()) {
        return;
    }
    if (LineText.isBlank(prefix)) {
        errors.add(ErrorCode.OPEN_BLANK_BEFORE, tag.getLine(), FIRST_LINE,
                String.format(WHITESPACE_BEFORE, prefix.length()));
    } else {
        errors.add(ErrorCode.OPEN_TEXT_BEFORE, tag.getLine(), FIRST_LINE,
                String.format(TEXT_BEFORE_SAME_LINE,
                        LineText.excerpt(LineText.trim(prefix))));
    }
}
```

Fuente: `validation/OpeningTagValidator.java`

```
/** Reporta 102 si la etiqueta no es exactamente "<?php", o 104 si está pegada al carácter siguiente. */
private static void checkLexeme(ValidationContext context, Token tag,
        ErrorCollector errors) {
    if (!EXPECTED_TAG.equals(tag.getLexeme())) {
        errors.add(ErrorCode.OPEN_MALFORMED, tag.getLine(), tag.getColumn(),
                String.format(MALFORMED, LineText.excerpt(tag.getLexeme())));
        return;
    }
    SourceLine line = context.getLine(tag.getLine());
    int after = tag.getColumn() - 1 + EXPECTED_TAG.length();
    // Si después de "<?php" hay un carácter que no es espacio ni tabulación, es error 104.
    if (after < line.getContent().length()
            && !LineText.isSpaceOrTab(line.getContent().charAt(after))) {
        int codePoint = line.getContent().codePointAt(after);
        errors.add(ErrorCode.OPEN_NO_SPACE_AFTER, tag.getLine(), after + 1,
                String.format(NO_SPACE_AFTER, new String(Character.toChars(codePoint))));
    }
}
```

Fuente: `validation/OpeningTagValidator.java`

Código	Se detecta en	Condición
106	`validate`	`source.hadBom()`; se reporta en la línea 1.
103	`validate`	No hay ningún `OPEN_TAG` (el texto no contiene `<?`).
100 / 101	`checkLinesBefore`	Por cada línea anterior a la de la etiqueta: con texto → 100; en blanco → 101.
100 / 101	`checkSameLinePrefix`	Lo que hay antes de la etiqueta en su línea: texto → 100; solo espacios → 101 con la cantidad.
102	`checkLexeme`	El lexema no es exactamente `<?php` (`<?PHP`, `<? php`, `<?=`…).
104	`checkLexeme`	`<?php` exacto pegado a un carácter que no es espacio/tab (ej. `<?phpecho`).
105	`validate`	Cada `OPEN_TAG` después del primero.

Las columnas se calculan con `tag.getColumn() - 1` (convertir columna desde 1 a índice desde 0) más la longitud de `<?php`.

ClosingTagValidator — códigos 110 a 116

```
/** El último CLOSE_TAG es "la" etiqueta de cierre; los anteriores se reportan como 116. */
@Override
public void validate(ValidationContext context, ErrorCollector errors) {
    List<Token> closeTags = context.tokensOfType(TokenType.CLOSE_TAG);
    if (closeTags.isEmpty()) {
        reportMissing(context, errors);
        return;
    }
    for (int i = 0; i < closeTags.size() - 1; i++) {
        Token earlier = closeTags.get(i);
        errors.add(ErrorCode.CLOSE_NOT_LAST, earlier.getLine(), earlier.getColumn(),
                String.format(NOT_LAST, earlier.getColumn()));
    }
    Token tag = closeTags.get(closeTags.size() - 1);
    checkRestOfLine(context, tag, errors);
    checkLineBreaksAfter(context, tag, errors);
    checkLinesAfter(context, tag, errors);
}
```

Fuente: `validation/ClosingTagValidator.java`

El **último** `CLOSE_TAG` es “la” etiqueta de cierre; los anteriores son error 116. Sobre la última se revisan tres cosas:

```
/** Reporta 113 por espacios o 115 por otro texto después de la etiqueta en su misma línea. */
private static void checkRestOfLine(ValidationContext context, Token tag,
        ErrorCollector errors) {
    int after = tag.getColumn() - 1 + CLOSE_TAG.length();
    String rest = context.getLine(tag.getLine()).getContent().substring(after);
    if (rest.isEmpty()) {
        return;
    }
    if (LineText.isBlank(rest)) {
        errors.add(ErrorCode.CLOSE_TRAILING_WHITESPACE, tag.getLine(), after + 1,
                String.format(WHITESPACE_AFTER, rest.length()));
    } else {
        errors.add(ErrorCode.CLOSE_TEXT_AFTER, tag.getLine(), after + 1,
                String.format(TEXT_AFTER, LineText.excerpt(LineText.trim(rest))));
    }
}
```

Fuente: `validation/ClosingTagValidator.java`

```
/** Reporta 114 una sola vez en la línea de la etiqueta, contando todos los saltos de línea hasta el
   final. */
private static void checkLineBreaksAfter(ValidationContext context, Token tag,
        ErrorCollector errors) {
    int lineBreaks = 0;
    for (int number = tag.getLine(); number <= context.getLineCount(); number++) {
        if (context.getLine(number).hasTerminator()) {
            lineBreaks++;
        }
    }
    if (lineBreaks > 0) {
        int column = tag.getColumn() + CLOSE_TAG.length();
        errors.add(ErrorCode.CLOSE_TRAILING_LINE_BREAKS, tag.getLine(), column,
                String.format(LINE_BREAKS_AFTER, lineBreaks));
    }
}
```

Fuente: `validation/ClosingTagValidator.java`

```
/** Reporta 112 por cada línea posterior a la etiqueta que tenga contenido no blanco. */
private static void checkLinesAfter(ValidationContext context, Token tag,
        ErrorCollector errors) {
    for (int number = tag.getLine() + 1; number <= context.getLineCount(); number++) {
        SourceLine line = context.getLine(number);
        if (!LineText.isBlank(line.getContent())) {
            errors.add(ErrorCode.CLOSE_CONTENT_AFTER, number, WHOLE_LINE,
                    String.format(CONTENT_AFTER,
                            LineText.excerpt(LineText.trim(line.getContent())),
```

```
                            LogFormat.formatLineNumber(number)));
        }
    }
}
```

Fuente: `validation/ClosingTagValidator.java`

```
/** Reporta 111 si la última línea no vacía parece "? >"; si no, 110 (mencionando un ?> dentro de
   comentario si lo hay). */
private static void reportMissing(ValidationContext context, ErrorCollector errors) {
    int lastNonBlank = lastNonBlankLine(context);
    if (lastNonBlank > 0) {
        String text = LineText.trim(context.getLine(lastNonBlank).getContent());
        if (isSpacedCloseTag(text)) {
            errors.add(ErrorCode.CLOSE_MALFORMED, lastNonBlank, WHOLE_LINE,
                    String.format(MALFORMED, LineText.excerpt(text)));
            return;
        }
    }
    String description = MISSING;
    int commentLine = lineOfCloseTagInComment(context);
    if (commentLine > 0) {
        description += String.format(MISSING_COMMENT_HINT,
                LogFormat.formatLineNumber(commentLine));
    }
    errors.add(ErrorCode.CLOSE_MISSING, context.getLineCount(), WHOLE_LINE, description);
}
```

Fuente: `validation/ClosingTagValidator.java`

Código	Condición
110	No existe `CLOSE_TAG`. Si había un `?>` dentro de un comentario, se agrega la pista “(the '?>' inside the comment on line NNNN is ignored).”
111	No existe `CLOSE_TAG` y la última línea no vacía es `?` + espacios + `>` (ej. `?  >`). Se reporta **en lugar** de 110.
112	Una línea posterior a la etiqueta tiene contenido no blanco (un error por línea).
113	Después de `?>` en su línea solo hay espacios/tabs (con la cantidad).
114	Hay saltos de línea desde la línea de `?>` hasta el final (se cuentan todos, incluidas líneas en blanco). Un solo error, en la línea de `?>`.
115	Después de `?>` en su línea hay otro texto.
116	Un `CLOSE_TAG` que no es el último.

> **Para recordar**
> Un archivo correcto **no tiene salto de línea después de** `?>`. Muchos editores (NetBeans incluido) agregan uno al guardar, y eso produce correctamente “Error 114 … 1 line break(s) found after the closing tag”.

AssignmentTarget y AssignmentTargetExtractor — qué hay a la izquierda del =

La validación de identificadores se aplica **solo** al destino de una asignación simple `=` (token `ASSIGN`). Primero hay que extraer ese destino.

```
/** Devuelve un destino por cada token ASSIGN, excepto las asignaciones de desestructuración. */
public List<AssignmentTarget> extract(List<Token> tokens) {
    List<AssignmentTarget> targets = new ArrayList<AssignmentTarget>();
    for (int i = 0; i < tokens.size(); i++) {
        if (tokens.get(i).is(TokenType.ASSIGN)) {
            AssignmentTarget target = extractAt(tokens, i);
            if (target != null) {
                targets.add(target);
            }
```

```
        }
    }
    return targets;
}
```

Fuente: `validation/AssignmentTargetExtractor.java`

```
/** Extrae el destino del "=" en la posición dada, o devuelve null si debe omitirse. */
private AssignmentTarget extractAt(List<Token> tokens, int assignIndex) {
    int index = assignIndex - 1;
    // Salta los espacios entre el destino y el "=".
    while (index >= 0 && tokens.get(index).is(TokenType.WHITESPACE)) {
        index--;
    }
    if (index >= 0 && isDestructuring(tokens, index)) {
        return null;
    }
    LinkedList<Token> collected = new LinkedList<Token>();
    // Junta tokens pegados hacia atrás; un "]" se salta junto con todo hasta su "[" para que $a[$i +
       1] sea un solo destino.
    while (index >= 0 && !isBoundary(tokens.get(index))) {
        int open = tokens.get(index).is(TokenType.RBRACKET)
                ? matchingOpenBracket(tokens, index) : index;
        for (int k = index; k >= open; k--) {
            collected.addFirst(tokens.get(k));
        }
        index = open - 1;
    }
    boolean constant = isConstKeyword(previousSignificant(tokens, index));
    return new AssignmentTarget(tokens.get(assignIndex), collected, constant);
}
```

Fuente: `validation/AssignmentTargetExtractor.java`

1. Desde el `=`, retrocede saltando espacios.
2. Si lo anterior es `)` (`list(...) =`) o un `[...]` sin nada pegado delante (`[$a, $b] =`): desestructuración → se omite (devuelve `null`).
3. Recolecta hacia atrás los tokens **pegados** (sin espacio entre ellos) hasta una frontera: espacio, salto, etiqueta, `; , ( ) { } [`, otro operador de asignación/comparación/lógico o un comentario. Un `]` salta hasta su `[` correspondiente para que `$a[$i + 1]` sea un solo destino.
4. Busca el token significativo anterior: si es `const`, el destino es un nombre de constante.
5. Crea `AssignmentTarget` con el `=`, los tokens recolectados y la bandera de constante.

```
/** Detecta desestructuración: ")" antes del "=" (list(...) =) o "[...]" sin nada pegado delante ([$a,
   $b] =). */
private static boolean isDestructuring(List<Token> tokens, int index) {
    Token last = tokens.get(index);
    if (last.is(TokenType.RPAREN)) {
        return true;
    }
    if (!last.is(TokenType.RBRACKET)) {
        return false;
    }
    int open = matchingOpenBracket(tokens, index);
    return open != index && (open == 0 || isBoundary(tokens.get(open - 1)));
}
```

Fuente: `validation/AssignmentTargetExtractor.java`

```
/** Indica si el token detiene la recolección: tipo frontera, comentario u operador compuesto. */
private static boolean isBoundary(Token token) {
    return BOUNDARY_TYPES.contains(token.getType())
            || token.getCategory() == TokenCategory.COMMENT
            || (token.is(TokenType.OTHER_OPERATOR)
                    && BOUNDARY_OPERATORS.contains(token.getLexeme()));
}
```

Fuente: `validation/AssignmentTargetExtractor.java`

```
/** Devuelve el destino crudo hasta el primer "[", "->" o "::". */
public String getBase() {
    int end = raw.length();
    for (String terminator : BASE_TERMINATORS) {
        int index = raw.indexOf(terminator);
        if (index >= 0 && index < end) {
            end = index;
        }
    }
    return raw.substring(0, end);
}
```

Fuente: `validation/AssignmentTarget.java`

`AssignmentTarget.getRaw()` concatena los lexemas (ej. `$-data` = tokens `$`, `-`, `data`). `getBase()` corta en el primer `[`, `->` o `::`: `$notas[$i]` se valida como `$notas`; `$obj->prop` como `$obj`.

IdentifierValidator — códigos 200 a 205

```
/** Valida cada destino y, si generó errores, marca sus tokens como reportados (así ñ en $ñame no da
   además un 401). */
@Override
public void validate(ValidationContext context, ErrorCollector errors) {
    for (AssignmentTarget target : extractor.extract(context.getTokens())) {
        int before = errors.count();
        check(target, errors);
        if (errors.count() > before) {
            for (Token token : target.getTokens()) {
                token.markReported();
            }
        }
    }
}
```

Fuente: `validation/IdentifierValidator.java`

```
/** Aplica las reglas a la base del destino ($notas en $notas[$i]); tras const no se exige $. */
private static void check(AssignmentTarget target, ErrorCollector errors) {
    String base = target.getBase();
    // 205: no hay identificador a la izquierda del "=".
    if (base.isEmpty()) {
        Token assign = target.getAssign();
        errors.add(ErrorCode.IDENT_EMPTY_TARGET, assign.getLine(), assign.getColumn(),
                String.format(EMPTY_TARGET, assign.getColumn()));
        return;
    }
    int nameStart = 0;
    if (!target.isConstant()) {
        // 203: la variable no empieza con $.
        if (!base.startsWith(DOLLAR)) {
            report(errors, target, ErrorCode.IDENT_MISSING_DOLLAR, 0,
                    String.format(MISSING_DOLLAR, LineText.excerpt(base)));
            return;
        }
        // 201: el $ no va seguido de ningún nombre.
        if (base.length() == DOLLAR.length()) {
            report(errors, target, ErrorCode.IDENT_INVALID_START, 0, DOLLAR_ONLY);
            return;
        }
        nameStart = DOLLAR.length();
    }
    checkFirstCharacter(target, base, nameStart, errors);
    checkLaterCharacters(target, base, nameStart, errors);
    // 204: el nombre coincide exactamente (con mayúsculas) con una palabra reservada.
    String name = base.substring(nameStart);
    if (ReservedWords.isReservedIdentifierName(name)) {
        report(errors, target, ErrorCode.IDENT_RESERVED_WORD, 0,
                String.format(RESERVED_WORD, LineText.excerpt(base), name));
    }
}
```

Fuente: `validation/IdentifierValidator.java`

```
/** Reporta 200 si el nombre empieza con dígito, o 201 si empieza con otro carácter no válido. */
private static void checkFirstCharacter(AssignmentTarget target, String base, int index,
        ErrorCollector errors) {
    int first = base.codePointAt(index);
    String shown = new String(Character.toChars(first));
    if (AsciiChars.isDigit(first)) {
        report(errors, target, ErrorCode.IDENT_STARTS_WITH_DIGIT, index,
                String.format(STARTS_WITH_DIGIT, LineText.excerpt(base), shown));
    } else if (!AsciiChars.isNameStart(first)) {
        report(errors, target, ErrorCode.IDENT_INVALID_START, index,
                String.format(INVALID_START, LineText.excerpt(base), shown));
    }
}
```

Fuente: `validation/IdentifierValidator.java`

```
/** Reporta 202 una vez por cada carácter posterior al primero que no sea letra, dígito o guion bajo. */
private static void checkLaterCharacters(AssignmentTarget target, String base,
        int nameStart, ErrorCollector errors) {
    int index = nameStart + Character.charCount(base.codePointAt(nameStart));
    while (index < base.length()) {
        int c = base.codePointAt(index);
        if (!AsciiChars.isNameChar(c)) {
            // Posición desde 1 dentro del texto crudo, contando puntos de código.
            int position = base.codePointCount(0, index) + 1;
            report(errors, target, ErrorCode.IDENT_INVALID_CHARACTER, index,
                    String.format(INVALID_CHARACTER, LineText.excerpt(base),
                            new String(Character.toChars(c)), position));
        }
        index += Character.charCount(c);
    }
}
```

Fuente: `validation/IdentifierValidator.java`

Orden	Código	Regla	Ejemplo
1	205	Destino vacío (nada a la izquierda del `=`). Termina.	`= 4;`
2	203	No empieza con `$` (si no es const). Termina.	`total = 7;`
3	201	Solo `$`, sin nombre. Termina.	`$ = 9;`
4	200	Primer carácter del nombre es dígito.	`$1variable`
4	201	Primer carácter es otro carácter inválido.	`$-data`, `$ñame`
5	202	Cada carácter posterior inválido (uno por ocurrencia, posición desde 1 en el texto crudo).	`$na@me` → '@' posición 4
6	204	El nombre coincide **exactamente** con una palabra reservada.	`$if`, `$else` (no `$If`)

Si un destino generó algún error (se compara `errors.count()` antes y después), todos sus tokens se marcan `reported`. La columna de cada error = columna del destino + desplazamiento del carácter problemático. Se trabaja con **puntos de código** (`codePointAt`, `Character.charCount`) para no partir caracteres fuera del BMP.

CommentValidator — códigos 300 a 304

```
/** Recorre cada línea llevando la profundidad de paréntesis/corchetes y el último token significativo
   para detectar 300–304. */
@Override
public void validate(ValidationContext context, ErrorCollector errors) {
    List<List<Token>> lines = groupByLine(context);
    int depth = 0;
    Token previous = null;
    for (int number = 1; number < lines.size(); number++) {
        List<Token> code = codeTokens(lines.get(number));
        // Una línea huérfana solo cuenta fuera de paréntesis y al inicio de una sentencia.
        if (depth == 0 && startsStatement(previous) && isOrphanText(code)) {
            reportOrphan(context, lines, number, code, errors);
        }
        checkSlashes(code, errors);
        checkBlockComments(lines.get(number), errors);
        for (Token token : code) {
            depth = Math.max(0, depth + depthChange(token));
            previous = token;
        }
    }
}
```

Fuente: `validation/CommentValidator.java`

Agrupa los tokens no-layout por línea (`groupByLine`) y recorre línea por línea llevando dos estados: `depth` (profundidad de `(`/`[`) y `previous` (último token de código, sin contar comentarios).

```
/** Indica si la línea es texto huérfano: empieza con una palabra no reservada y solo tiene palabras,
   números o desconocidos. */
private static boolean isOrphanText(List<Token> code) {
    if (code.isEmpty() || !code.get(0).is(TokenType.IDENTIFIER)) {
        return false;
    }
    for (Token token : code) {
        if (!ORPHAN_PARTS.contains(token.getType())) {
            return false;
        }
    }
    return true;
}
```

Fuente: `validation/CommentValidator.java`

```
/** Reporta 300 si la línea anterior termina en comentario de línea (parece continuarlo) o 301 en otro
   caso. */
private static void reportOrphan(ValidationContext context, List<List<Token>> lines,
        int number, List<Token> code, ErrorCollector errors) {
    Token first = code.get(0);
    Token last = code.get(code.size() - 1);
    String text = LineText.excerpt(context.getLine(number).getContent().substring(
            first.getColumn() - 1, last.getColumn() - 1 + last.getLexeme().length()));
    List<Token> previousLine = lines.get(number - 1);
    boolean afterInlineComment = !previousLine.isEmpty()
            && previousLine.get(previousLine.size() - 1).is(TokenType.INLINE_COMMENT);
    if (afterInlineComment) {
        errors.add(ErrorCode.COMMENT_CONTINUATION, number, first.getColumn(), String.format(
                CONTINUATION, text, LogFormat.formatLineNumber(number - 1)));
    } else {
        errors.add(ErrorCode.COMMENT_UNMARKED_TEXT, number, first.getColumn(),
                String.format(UNMARKED_TEXT, text));
    }
}
```

Fuente: `validation/CommentValidator.java`

Una **línea huérfana** es texto que no es PHP ni comentario: profundidad 0, empieza una sentencia (el token previo es `;`, `{`, `}`, `<?php` o no hay), su primer token es una palabra **no reservada** y solo contiene palabras, números o caracteres desconocidos (por eso no puede tener `;`, `(` ni `::`). Si la línea anterior termina en comentario de línea → 300 (“parece continuar el comentario”); si no → 301.

```
/** Reporta 302 por un "/" al inicio de la línea o tras ; { } en la misma línea; un "/" tras un
   operando ($a / $b) es división. */
private static void checkSlashes(List<Token> code, ErrorCollector errors) {
    for (int i = 0; i < code.size(); i++) {
        Token token = code.get(i);
        if (!SLASH.equals(token.getLexeme())) {
            continue;
        }
        boolean statementStart = i == 0
                || STATEMENT_STARTERS.contains(code.get(i - 1).getType());
        if (statementStart && !isGluedToMarker(code, i)) {
            errors.add(ErrorCode.COMMENT_MALFORMED_MARKER, token.getLine(),
                    token.getColumn(), String.format(MALFORMED_MARKER, token.getColumn()));
        }
    }
}
```

Fuente: `validation/CommentValidator.java`

```
/** Indica si el "/" en la posición dada va pegado inmediatamente a "/", "*" o "=". */
private static boolean isGluedToMarker(List<Token> code, int index) {
    if (index + 1 >= code.size()) {
        return false;
    }
    Token slash = code.get(index);
    Token next = code.get(index + 1);
    return next.getLine() == slash.getLine() && next.getColumn() == slash.getColumn() + 1
            && NOT_A_MARKER_AFTER_SLASH.indexOf(next.getLexeme().charAt(0)) >= 0;
}
```

Fuente: `validation/CommentValidator.java`

Error 302: una `/` al inicio de la línea o justo después de `;`, `{`, `}` (o `<?php`) que no esté pegada a `/`, `*` o `=`. Por eso la división `$suma_notas / $cantidad_notas` (línea 22 de codigo.php) **no** es error: la `/` viene después de un operando.

```
/** Reporta 303 por un comentario de bloque cerrado que abarca varias líneas y 304 por uno que nunca se
   cierra. */
private static void checkBlockComments(List<Token> lineTokens, ErrorCollector errors) {
    for (Token token : lineTokens) {
        if (token.is(TokenType.UNTERMINATED_COMMENT)) {
            errors.add(ErrorCode.COMMENT_UNTERMINATED, token.getLine(), token.getColumn(),
                    String.format(UNTERMINATED, token.getColumn()));
        } else if (token.is(TokenType.BLOCK_COMMENT) && token.getEndLine() > token.getLine()) {
            errors.add(ErrorCode.COMMENT_MULTILINE_BLOCK, token.getLine(), token.getColumn(),
                    String.format(MULTILINE_BLOCK, token.getColumn(),
                            LogFormat.formatLineNumber(token.getLine()),
                            LogFormat.formatLineNumber(token.getEndLine())));
        }
    }
}
```

Fuente: `validation/CommentValidator.java`

LexemeValidator — códigos 400 a 402

```
/** Se ejecuta al final y omite tokens ya reportados (así ñ en $ñame = solo da el error 201, no un
   401). */
@Override
public void validate(ValidationContext context, ErrorCollector errors) {
    for (Token token : context.getTokens()) {
        if (!token.isReported()) {
            check(token, errors);
        }
    }
}
```

Fuente: `validation/LexemeValidator.java`

```
/** Reporta el error que corresponde al tipo del token y lo marca como reportado. */
private static void check(Token token, ErrorCollector errors) {
    int line = token.getLine();
```

```
    int column = token.getColumn();
    String lexeme = token.getLexeme();
    switch (token.getType()) {
        case UNTERMINATED_STRING:
            errors.add(ErrorCode.LEXEME_UNTERMINATED_STRING, line, column,
                    String.format(UNTERMINATED_STRING, lexeme.charAt(0), column));
            break;
        case UNKNOWN:
            errors.add(ErrorCode.LEXEME_UNRECOGNIZED_CHARACTER, line, column,
                    String.format(UNRECOGNIZED, lexeme, lexeme.codePointAt(0), column));
            break;
        case MALFORMED_NUMBER:
            errors.add(ErrorCode.LEXEME_MALFORMED_NUMBER, line, column,
                    String.format(MALFORMED_NUMBER, LineText.excerpt(lexeme), column));
            break;
        default:
            return;
    }
    token.markReported();
}
```

Fuente: `validation/LexemeValidator.java`

Corre al final y omite los tokens ya reportados. `UNTERMINATED_STRING` → 400 (indica la comilla y la columna); `UNKNOWN` → 401 con el código Unicode `U+XXXX`; `MALFORMED_NUMBER` → 402.

12. Paquetes de soporte: errors, symbols y report

ErrorCode — el catálogo

```
/** Catálogo de todos los códigos de error léxico del PRD §5 con su número y categoría; modelo de
   errores de la Etapa 4. */
public enum ErrorCode {

    // 5.1 Etiqueta de apertura <?php (códigos 100–106).
    OPEN_TEXT_BEFORE(100, Category.OPENING_TAG),
    OPEN_BLANK_BEFORE(101, Category.OPENING_TAG),
    OPEN_MALFORMED(102, Category.OPENING_TAG),
    OPEN_MISSING(103, Category.OPENING_TAG),
    OPEN_NO_SPACE_AFTER(104, Category.OPENING_TAG),
    OPEN_ADDITIONAL(105, Category.OPENING_TAG),
    OPEN_BOM(106, Category.OPENING_TAG),

    // 5.2 Etiqueta de cierre ?> (códigos 110–116).
    CLOSE_MISSING(110, Category.CLOSING_TAG),
    CLOSE_MALFORMED(111, Category.CLOSING_TAG),
    CLOSE_CONTENT_AFTER(112, Category.CLOSING_TAG),
    CLOSE_TRAILING_WHITESPACE(113, Category.CLOSING_TAG),
    CLOSE_TRAILING_LINE_BREAKS(114, Category.CLOSING_TAG),
    CLOSE_TEXT_AFTER(115, Category.CLOSING_TAG),
    CLOSE_NOT_LAST(116, Category.CLOSING_TAG),

    // 5.3 Identificadores (códigos 200–205).
    IDENT_STARTS_WITH_DIGIT(200, Category.IDENTIFIER),
    IDENT_INVALID_START(201, Category.IDENTIFIER),
    IDENT_INVALID_CHARACTER(202, Category.IDENTIFIER),
    IDENT_MISSING_DOLLAR(203, Category.IDENTIFIER),
    IDENT_RESERVED_WORD(204, Category.IDENTIFIER),
    IDENT_EMPTY_TARGET(205, Category.IDENTIFIER),

    // 5.4 Comentarios (códigos 300–304).
    COMMENT_CONTINUATION(300, Category.COMMENT),
    COMMENT_UNMARKED_TEXT(301, Category.COMMENT),
    COMMENT_MALFORMED_MARKER(302, Category.COMMENT),
    COMMENT_MULTILINE_BLOCK(303, Category.COMMENT),
    COMMENT_UNTERMINATED(304, Category.COMMENT),

    // 5.5 Otros lexemas (códigos 400–402).
```

Fuente: `errors/ErrorCode.java` *(constantes)*

Enum con 29 constantes; cada una tiene su número (`getCode()`) y su `Category` (`OPENING_TAG`, `CLOSING_TAG`, `IDENTIFIER`, `COMMENT`, `LEXEME`). Usar un enum en vez de enteros sueltos evita errores de tipeo y documenta todos los códigos en un solo lugar.

LexicalError — un error

```
/** Formatea el error tal como aparece en la bitácora: "Error 200. Line 0005. ..." (sin terminador). */
public String toLogLine() {
    return LOG_PREFIX + code.getCode()
            + LINE_LABEL + LogFormat.formatLineNumber(line)
            + DESCRIPTION_SEPARATOR + description;
}
```

Fuente: `errors/LexicalError.java`

```
/** Ordena por línea, luego por columna y luego por código numérico. */
@Override
public int compareTo(LexicalError other) {
    if (line != other.line) {
        return Integer.compare(line, other.line);
    }
    if (column != other.column) {
        return Integer.compare(column, other.column);
    }
    return Integer.compare(code.getCode(), other.code.getCode());
```

```
}
```

Fuente: `errors/LexicalError.java`

Inmutable, implementa `Comparable` (orden: línea, columna, código). La columna 0 significa "toda la línea", por eso esos errores quedan primero dentro de su línea. También redefine `equals`/`hashCode` (coherentes entre sí) y `toString`.

ErrorCollector

```
/** Devuelve una copia de solo lectura de los errores ordenados por línea, columna y código. */
public List<LexicalError> getSorted() {
    List<LexicalError> sorted = new ArrayList<LexicalError>(errors);
    Collections.sort(sorted);
    return Collections.unmodifiableList(sorted);
}
```

Fuente: `errors/ErrorCollector.java`

Guarda los errores en orden de llegada; `add(...)` tiene un atajo que construye el `LexicalError`. `getSorted()` devuelve una copia ordenada con `Collections.sort` y no modificable. `count()` lo usa `IdentifierValidator` para saber si un destino generó errores.

SymbolTable y SymbolEntry

```
/** Construye la tabla con los tokens VARIABLE de nombre bien formado (los mal formados son errores, no
   símbolos). */
public static SymbolTable build(List<Token> tokens) {
    SymbolTable table = new SymbolTable();
    for (Token token : tokens) {
        if (token.is(TokenType.VARIABLE) && hasWellFormedName(token.getLexeme())) {
            table.record(token.getLexeme(), token.getLine());
        }
    }
    return table;
}
```

Fuente: `symbols/SymbolTable.java`

```
/** Registra una aparición del identificador: crea la entrada o incrementa su contador. */
public void record(String name, int line) {
    SymbolEntry entry = entries.get(name);
    if (entry == null) {
        entries.put(name, new SymbolEntry(name, line));
    } else {
        entry.addOccurrence();
    }
}
```

Fuente: `symbols/SymbolTable.java`

Usa un `LinkedHashMap<String, SymbolEntry>`: la **clave es el texto exacto** (por eso `$total` y `$Total` son entradas distintas) y el `LinkedHash` conserva el orden de primera aparición. Solo se registran variables bien formadas (`$` + letra o _). `SymbolEntry` guarda nombre, primera línea y contador (`addOccurrence` es de visibilidad de paquete: solo la tabla lo incrementa).

ConsoleReporter

Única clase de impresión (además de `Application`). Recibe un `PrintStream`, por lo que podría redirigirse a otro destino. Métodos principales:

Método	Qué imprime
`printHeader`	"PHP Lexical Analyzer".
`printIgnored`	"Ignored : x.PHP (motivo)" por cada candidato rechazado.

Método	Qué imprime
`printFileSummary`	Ruta, líneas, tokens, errores y ruta de la bitácora.
`printSymbols`	Tabla de símbolos con ancho de columna calculado según el nombre más largo.
`printTokens`	Tabla LINE/COL/TYPE/CATEGORY/LEXEME y si el invariante se cumple. `quote` escapa `\r`, `\n`, `\t`, `\\` y controles.
`printWrongExtension`	Mensaje exacto del PRD: `ERROR: 'x' does not have the required .php extension; the file was not processed.`
`printNoPhpFound`, `printFileNotFound`, `printReadError`, `printLogWriteError`, `printSearchError`, `printUsageError`	Mensajes de error de consola con el detalle.

```
/** Imprime la tabla de símbolos, un identificador por fila, para mostrar que $total y $Total son
   distintos. */
public void printSymbols(SymbolTable symbols) {
    out.println(SYMBOLS_TITLE);
    if (symbols.size() == 0) {
        out.println(NO_SYMBOLS);
        return;
    }
    // Calcula el ancho de la columna de nombres según el identificador más largo.
    int width = 0;
    for (SymbolEntry entry : symbols.getEntries()) {
        width = Math.max(width, entry.getName().length());
    }
    String format = SYMBOL_ROW_FORMAT.replace("{width}",
            String.valueOf(width + SYMBOL_NAME_PADDING));
    for (SymbolEntry entry : symbols.getEntries()) {
        out.println(String.format(format, entry.getName(),
                LogFormat.formatLineNumber(entry.getFirstLine()), entry.getOccurrences()));
    }
}
```

Fuente: `report/ConsoleReporter.java`

13. Pruebas: FixtureRunner y fixtures

Como está prohibido JUnit, las pruebas son un programa con `main` en `test/compilador/FixtureRunner.java`. Usa **el mismo** `FileAnalyzer` que el jar.

```
/** Copia el fixture, lo analiza, aplica las tres verificaciones e imprime el resultado; devuelve true
   si todas pasan. */
private boolean runFixture(Path fixture, Path expectedDir, Path workDir) throws IOException {
    String name = fixture.getFileName().toString();
    Path copy = Files.copy(fixture, workDir.resolve(name), StandardCopyOption.REPLACE_EXISTING);
    List<String> failures = new ArrayList<String>();
    try {
        SourceFile source = analyzer.read(copy);
        AnalysisResult result = analyzer.analyze(source);
        byte[] log = Files.readAllBytes(analyzer.writeLog(result));
        Path expected = expectedDir.resolve(baseName(name) + LogFormat.LOG_FILE_SUFFIX);
        addIfFailed(failures, checkExpectedLog(log, expected, source.getCharset()));
        addIfFailed(failures, checkNumberedCopy(log, source, Files.readAllBytes(fixture)));
        addIfFailed(failures, checkLexemeInvariant(result));
    } catch (IOException e) {
        failures.add("I/O error: " + e);
    }
    System.out.println((failures.isEmpty() ? "PASS " : "FAIL ") + name);
    for (String failure : failures) {
        System.out.println("     " + failure);
    }
    return failures.isEmpty();
}
```

Fuente: `test/compilador/FixtureRunner.java`

Verificación	Qué comprueba
1. `checkExpectedLog`	La bitácora generada es **idéntica byte a byte** a `test-fixtures/expected/<nombre>-errors.log`. Si no, muestra la primera línea distinta.
2. `checkNumberedCopy`	Cada línea copiada sin su prefijo `NNNN` es igual a la línea fuente con su terminador, y juntas reconstruyen los bytes originales (sin BOM).
3. `checkLexemeInvariant`	Concatenar los lexemas = texto fuente.

Cada fixture se copia a una carpeta temporal (para no ensuciar `test-fixtures`), se analiza, se aplican las tres verificaciones y se imprime `PASS`/`FAIL`. Al final: “N passed, M failed” y código de salida 1 si alguno falló.

Los 16 fixtures

Fixture	Propósito	Códigos esperados
`valid_codigo.php`	Archivo del profesor sin salto final	ninguno
`valid_codigo_crlf.php`	Mismo archivo con CRLF; la bitácora conserva CRLF	ninguno
`identifiers.php`	Reglas de identificadores (PRD §8.1)	200, 201, 202, 203, 204
`open_leading_ws.php`	`<?php`	101
`open_content_before.php`	`echo "";` / línea en blanco / `<?php`	100, 101
`open_malformed.php`	`<?PHP`	102
`open_missing.php`	Sin etiqueta	103
`close_missing.php`	Sin `?>`	110
`close_trailing_newline.php`	Termina en `?>\n\n`	114 (n = 2)
`close_trailing_space.php`	`?>` + 2 espacios	113

Fixture	Propósito	Códigos esperados
`close_content_after.php`	`?>` / `echo "hello";`	114, 112
`comments.php`	Comentarios (PRD §8.2)	300, 301, 302, 303
`bom.php`	BOM + código válido	106
`empty.php`	0 bytes	103, 110
`latin1.php`	Archivo ISO-8859-1 con `$año =` (respaldo de charset)	202 (ñ en posición 3)
`tabs_trailing_spaces.php`	Tabs y espacios al final de línea se conservan byte a byte	ninguno

identifiers.php: entrada y errores esperados

```
<?php
$variable = 1;
$_data = 2;
$name1 = 3;
$1variable = 4;
$-data = 5;
$if = 10;
$else = true;
$1a = 1; $for = 2;
$If = 3;
$na@me = 6;
total = 7;
const MAX = 10;
$notas[0] = 5;
$x == 5;
?>
```

```
Error 200. Line 0005. Invalid identifier '$1variable': it starts with the digit '1' after '$'; ...
Error 201. Line 0006. Invalid identifier '$-data': character '-' after '$' is not allowed; ...
Error 204. Line 0007. Invalid identifier '$if': 'if' is a reserved word and cannot be used as ...
Error 204. Line 0008. Invalid identifier '$else': 'else' is a reserved word and cannot be used ...
Error 200. Line 0009. Invalid identifier '$1a': it starts with the digit '1' after '$'; ...
Error 204. Line 0009. Invalid identifier '$for': 'for' is a reserved word and cannot be used ...
Error 202. Line 0011. Invalid identifier '$na@me': character '@' at position 4 is not allowed; ...
Error 203. Line 0012. Invalid identifier 'total': the assignment target does not start with '$'; ...
```

Sección de errores esperada (recortada con "..." para que quepa).

Observa: la línea 9 tiene **dos** errores (dos asignaciones), ordenados por columna; `$If` (línea 10) no es error por la sensibilidad a mayúsculas; `const MAX` (13) es válido; `$notas[0]` (14) se valida como `$notas`; `$x == 5` (15) no es asignación.

comments.php: entrada y errores

```
<?php
$var = 10; // if while echo
# $echo another valid comment
$url = "http://example.com # not a comment";
$a = 1; // comment
continues here
$b = 2;
stray words
$c = 3; / bad comment
/* block
comment */
$d = 4; /* ok on one line */
?>
```

```
Error 300. Line 0006. Text 'continues here' appears to continue the inline comment of line 0005; ...
Error 301. Line 0008. Text 'stray words' is not valid PHP code and is not marked as a comment; ...
Error 302. Line 0009. Malformed comment marker '/' at column 9; inline comments must start ...
Error 303. Line 0010. Comment starting at column 1 spans lines 0010 to 0011; comments must not ...
```

14. Trazas paso a paso

Practica estas trazas: son la mejor forma de demostrar que entiendes cómo cooperan las clases.

Traza 1: tokenizar la línea 19 de codigo.php

```
    $suma_notas = $suma_notas + $notas[$i];
```

El lexer está en modo PHP (ya pasó `<?php`). En cada vuelta de `tokenize`, `findPhpPattern` prueba los patrones en orden y gana el primero cuyo `canStart` es verdadero:

Col	Carácter actual	Patrón ganador	Lexema	Tipo final (Etapa 2)
1	espacio	WhitespacePattern	`' '`	WHITESPACE
5	`$`	VariablePattern (DOLLAR→NAME…→DONE)	`$suma_notas`	VARIABLE
16	espacio	WhitespacePattern	`' '`	WHITESPACE
17	`=`	OperatorPattern (`=` es el más largo que coincide: no hay `==`)	`=`	ASSIGN (tabla del clasificador)
18	espacio	WhitespacePattern	`' '`	WHITESPACE
19	`$`	VariablePattern	`$suma_notas`	VARIABLE
31	`+`	OperatorPattern (`++` y `+=` no coinciden)	`+`	ARITHMETIC_OPERATOR
33	`$`	VariablePattern (se detiene en `[`)	`$notas`	VARIABLE
39	`[`	OperatorPattern	`[`	LBRACKET
40	`$`	VariablePattern	`$i`	VARIABLE
42	`]`	OperatorPattern	`]`	RBRACKET
43	`;`	OperatorPattern	`;`	SEMICOLON
44	`\n`	NewlinePattern	`\n`	NEWLINE (la línea pasa a 20)

Esto coincide exactamente con la salida real de `--tokens` (filas `0019`). Luego la Etapa 4: `AssignmentTargetExtractor` encuentra el `ASSIGN` de la columna 17, retrocede saltando el espacio, recolecta `$suma_notas` y se detiene en el `WHITESPACE` de la columna 1. Destino = `$suma_notas`, válido → sin errores.

Traza 2: validar $na@me = 6;

1. Tokens: `$na` (VARIABLE; el autómata se detiene en `@`), `@` (OTHER_OPERATOR), `me` (IDENTIFIER), espacio, `=` (ASSIGN), espacio, `6`, `;`.
2. Extractor: desde `=` salta el espacio; recolecta `me`, `@`, `$na` (pegados, ninguno es frontera); se detiene en el NEWLINE anterior. Raw = `$na@me`, base = `$na@me` (no hay `[`, `->`, `::`).
3. `check`: no está vacío (no 205); empieza con `$` (no 203); tiene nombre (no 201 de `$` solo).
4. `checkFirstCharacter`: `n` es letra → sin error.
5. `checkLaterCharacters`: recorre `a`, `@`, `m`, `e`. `@` no es `[A-Za-z0-9_]` → **202** con posición `codePointCount(0, 3) + 1 = 4`.
6. 204: `na@me` no es palabra reservada.
7. Como hubo errores, `$na`, `@` y `me` se marcan `reported`. `LexemeValidator` no los volverá a reportar.

```
Error 202. Line 0011. Invalid identifier '$na@me': character '@' at position 4 is not allowed; only
letters, digits (0-9) and underscores (_) may follow the first character.
```

Traza 3: $ñame = 1; (deduplicación)

Tokens: `$` (VARIABLE con nombre vacío: `ñ` no es ASCII y detiene el autómata), `ñ` (**UNKNOWN**, ningún patrón lo acepta), `ame` (IDENTIFIER). Raw = `$ñame`. El primer carácter del nombre es `ñ` → **201**. Los tres tokens se marcan `reported`, así que `LexemeValidator` omite el `UNKNOWN` y **no** aparece un 401 adicional. Salida real:

```
Error 201. Line 0002. Invalid identifier '$ñame': character 'ñ' after '$' is not allowed; an
identifier must start with a letter (a-z, A-Z) or underscore (_).
```

Traza 4: comentario que "continúa" en la línea siguiente

```
5: $a = 1; // comment
6: continues here
7: $b = 2;
```

1. Línea 5: códigos `$a`, `=`, `1`, `;` (el `INLINE_COMMENT` está en la lista de la línea pero se excluye de `codeTokens`). Al terminar, `previous = ;` y `depth = 0`.
2. Línea 6: código = `continues` (IDENTIFIER), `here` (IDENTIFIER). `depth == 0`, `startsStatement(;)` verdadero, `isOrphanText` verdadero (primer token IDENTIFIER no reservado, todos son palabras).
3. `reportOrphan`: el último token no-layout de la línea 5 es `INLINE_COMMENT` → **300** "appears to continue the inline comment of line 0005".
4. Si la línea 5 no terminara en comentario, sería **301**.

Traza 5: construcción de la bitácora

Archivo de 3 líneas `<?php\n$1x = 1;\n?>` (sin salto final):

```
0001 <?php\n                      <- prefijo + contenido + terminador original
0002 $1x = 1;\n
0003 ?>                         <- última línea SIN terminador
\n                              <- separador agregado (la última línea no tenía)
\n                              <- línea en blanco obligatoria
Error 200. Line 0002. Invalid identifier '$1x': it starts with the digit '1' after '$'; ...\n
```

El separador usado es el primer terminador del archivo (`\n`).

Traza 6: números

Entrada	Recorrido del autómata	Resultado
`42`	START→INTEGER→INTEGER→DONE	INTEGER_LITERAL
`3.14`	START→INTEGER→FRACTION_START→FRACTION→FRACTION→DONE	FLOAT_LITERAL
`.5`	START(`.`)→FRACTION_START→FRACTION→DONE	FLOAT_LITERAL
`1_000`	INTEGER acepta `_` seguido de dígito	INTEGER_LITERAL
`2e10`	INTEGER→EXPONENT	FLOAT_LITERAL
`0x1F`	START (prefijo 0x + dígito hex)→HEX→DONE	INTEGER_LITERAL
`12abc`	INTEGER→DONE; luego `a` es carácter de nombre → consume `abc`	MALFORMED_NUMBER → 402

15. Catálogo completo de errores

Formato **siempre**: `Error <código>. Line <NNNN>. <descripción específica>`. Los textos citados se recortan a 40 caracteres + `...`. Varios errores en una línea se reportan por separado.

Código	Validador	Cuándo	Ejemplo que lo produce
100	Opening	Texto antes de `<?php` (líneas anteriores o misma línea)	`echo "";` en la línea 1
101	Opening	Línea en blanco o espacios antes de `<?php`	`<?php`
102	Opening	Etiqueta no exactamente `<?php`	`<?PHP`, `<? php`, `<?`
103	Opening	No hay `<?` en el archivo (línea 1)	archivo vacío
104	Opening	`<?php` pegado a otro carácter	`<?phpecho 1;`
105	Opening	Segunda etiqueta de apertura	`?>` … `<?php` de nuevo
106	Opening	BOM UTF-8 al inicio	guardar “UTF-8 con BOM”
110	Closing	No existe `?>` (última línea)	falta `?>`
111	Closing	`? >` mal escrito en la última línea no vacía	`? >`
112	Closing	Línea con contenido después de `?>`	`echo "hello";` tras `?>`
113	Closing	Espacios después de `?>`	`?>`
114	Closing	Saltos de línea después de `?>`	`?>\n`
115	Closing	Texto después de `?>` en su línea	`?> hola`
116	Closing	Un `?>` que no es el último	dos bloques PHP
200	Identifier	Nombre empieza con dígito	`$1variable = 4;`
201	Identifier	Nombre empieza con carácter inválido, o `$` solo	`$-data`, `$ñame`, `$ = 9`
202	Identifier	Carácter inválido después del primero (uno por ocurrencia)	`$na@me = 6;`
203	Identifier	Destino sin `$`	`total = 7;`
204	Identifier	Nombre = palabra reservada (exacto)	`$if = 10;`
205	Identifier	Nada a la izquierda del `=`	`= 4;`
300	Comment	Texto huérfano tras línea con comentario `//`	`// x / continues here`
301	Comment	Texto huérfano en otro lugar	`stray words`
302	Comment	`/` suelta al inicio de sentencia	`$c = 3; / bad`
303	Comment	`/* */` en varias líneas	`/* block / comment */`
304	Comment	`/*` nunca cerrado	`/* abierto` hasta EOF
400	Lexeme	Cadena sin cerrar	`$s = "abierta;`
401	Lexeme	Carácter no reconocido (con U+hex)	€ → U+20AC
402	Lexeme	Número seguido de letras	`12abc`

Salida real con varios errores a la vez

```
<?php
$ñame = 1;
$x = 12abc;
$s = "abierta;
$y = 5 € 2;
$a+=1; $b=2;
$obj->prop = 3;
[$p, $q] = [1, 2];
= 4;
$ = 9;
const 9MAX = 1;
$If = 1;
// fin ?>
```

Archivo de prueba (con salto final después de la línea 13).

```
Error 201. Line 0002. Invalid identifier '$ñame': character 'ñ' after '$' is not allowed; ...
Error 402. Line 0003. Malformed numeric literal '12abc' at column 6; a number cannot be ...
Error 400. Line 0004. String literal opened with " at column 6 is never closed.
Error 401. Line 0005. Unrecognized character '€' (U+20AC) at column 8.
Error 205. Line 0009. Assignment operator '=' at column 1 has no identifier on its left side.
Error 201. Line 0010. Invalid identifier '$': the dollar sign is not followed by a name.
Error 200. Line 0011. Invalid identifier '9MAX': it starts with the digit '9' after '$'; ...
Error 110. Line 0013. Closing tag '?>' not found; the file must end with '?>'. (the '?>'
          inside the comment on line 0013 is ignored).
```

Observa lo que **no** es error: `$a+=1` (no es `ASSIGN`), `$b=2` (sin espacios, válido), `$obj->prop` (base `$obj`), `[$p, $q] =` (desestructuración), `$If` (mayúscula), la línea 5 tras la cadena sin cerrar (la recuperación la dejó analizar normalmente) y `?>` dentro del comentario (ignorado → 110).

16. Decisiones de diseño y limitaciones

Decisión	Justificación
Autómatas escritos a mano, sin regex ni generadores	Requisito C2/C3 y para hacer visible la etapa de “determinación de patrones”. Cada patrón documenta su regex equivalente.
Se conservan espacios, saltos y comentarios como tokens	La extracción del destino de `=` y las reglas de comentarios dependen de la **adyacencia**; además permite el invariante lexemas = fuente.
Patrones permisivos + validación posterior	`VariablePattern` acepta `$1x`, `OpenTagPattern` acepta `<?PHP`: así la Etapa 4 da mensajes **específicos** en vez de un genérico “error de sintaxis”.
Identificadores sensibles a mayúsculas	`$if` es error, `$If` no; `$total` ≠ `$Total` en la tabla de símbolos. Igual que PHP con variables.
Palabras sueltas clasificadas sin distinguir mayúsculas	PHP trata `ECHO` y `echo` igual.
“Include” del enunciado = `include`	Uniformidad: todas las reservadas en minúscula.
`?>` dentro de comentarios se ignora	El enunciado dice que el contenido de los comentarios se ignora por completo. En PHP real `?>` sí cerraría el bloque: desviación documentada.
Recuperación de cadenas sin cerrar al final de la línea	Evita que una comilla olvidada se trague el `?>` y genere errores en cascada.
Sin `<?` → todo en modo PHP	Para que las demás validaciones sigan funcionando aunque falte la etiqueta (103).
Separador de bitácora en una constante	`LogFormat.NUMBER_SEPARATOR = " "`: si el profesor lo quiere pegado, basta cambiarlo a `""`.
BOM detectado, quitado y reportado (106)	Es un marcador invisible de codificación, no un carácter del programa; no se reproduce en la bitácora.
UTF-8 estricto con respaldo ISO-8859-1	Archivos guardados en Latin-1 (común en Windows) se leen sin “caracteres raros”; la bitácora se escribe con el mismo charset.
Columna 0 = “toda la línea”	Errores que no tienen un carácter concreto (101 por línea en blanco, 110, 112) quedan primero al ordenar.
Bandera `reported` y orden fijo de validadores	Un mismo problema no genera dos errores (201 vs 401).

Limitaciones (fuera de alcance, PRD §10)

- Heredoc/nowdoc (`<<<EOT`) no se reconocen.
- No se analiza la interpolación dentro de cadenas con comillas dobles: la cadena es un solo token.
- No hay análisis sintáctico ni semántico (variables no declaradas, paréntesis balanceados, etc.).
- Varios bloques PHP intercalados con HTML solo se reportan mediante 105/116.
- Identificadores Unicode no se aceptan (`$año` es error).

Detalles finos que conviene conocer (por si te los señalan)

- En una constante (`const 9MAX = 1;`) los errores 200/201 reutilizan la plantilla del PRD, que dice “after '$'” aunque una constante no lleva `$`. Es literal de la plantilla del PRD.
- En la pista del error 110 la frase queda “…must end with '?>'. (the '?>' inside…)”: el PRD indica **agregar** ese texto al final de la plantilla, por eso hay punto antes del paréntesis.
- La tabla de símbolos registra `$na` al encontrar `$na@me` (el token VARIABLE es `$na`), aunque el destino completo sea inválido.
- Las columnas cuentan unidades UTF-16 (`char` de Java): un emoji ocupa 2 columnas.

17. Escenarios: ¿qué pasa si cambia algo en el código?

El profesor puede proponer un cambio y preguntar el efecto. La estrategia: (1) ubicar la clase y la etapa, (2) seguir el flujo de datos, (3) mencionar qué fixture fallaría.

Escenario 1. ¿Qué pasa si... se cambia `LogFormat.NUMBER_SEPARATOR` de `" "` a `""`?
La bitácora queda `0001<?php`. Solo cambia esa constante: `ErrorLogWriter` y `FixtureRunner` la usan. `checkNumberedCopy` sigue pasando, pero `checkExpectedLog` falla en todos los fixtures hasta regenerar las bitácoras esperadas.

Escenario 2. ¿Qué pasa si... se cambia `LINE_NUMBER_WIDTH` a 3?
El formato pasa a `%03d`: `001`. También cambian los números de línea dentro de los mensajes de error (`Line 001`), porque `LexicalError.toLogLine` usa el mismo `formatLineNumber`.

Escenario 3. ¿Qué pasa si... se mueve `OperatorPattern` antes de `InlineCommentPattern` en `PatternRegistry`?
`//` ya no sería comentario: `/` y `/` serían dos operadores y el texto del comentario se tokenizaría (`if`, `while`, `echo` como KEYWORD). Aparecerían errores 302 y 301 falsos y el fixture `comments.php` fallaría. `#` seguiría funcionando porque no es operador.

Escenario 4. ¿Qué pasa si... se mueve `CloseTagPattern` después de `OperatorPattern`?
`?>` se reconocería como `?` y `>`: nunca habría `CLOSE_TAG`, el lexer nunca volvería a modo INLINE y todo archivo daría el error 110. `valid_codigo.php` fallaría.

Escenario 5. ¿Qué pasa si... se mueve `NumberPattern` después de `OperatorPattern`?
`.5` se partiría en `.` (CONCAT) y `5`. Los enteros no cambian porque ningún operador empieza con dígito.

Escenario 6. ¿Qué pasa si... se cambia el orden entre `WordPattern` y `VariablePattern`?
Nada: sus caracteres iniciales son disjuntos (`$` no es inicio de nombre). Buena respuesta para mostrar que entiendes por qué el orden importa solo cuando dos patrones pueden empezar igual.

Escenario 7. ¿Qué pasa si... `IDENTIFIER_CHECK_CASE_SENSITIVE` pasa a `false`?
`$If`, `$ECHO`, `$Class` darían error 204. En `identifiers.php` aparecería un 204 en la línea 10 y el fixture fallaría.

Escenario 8. ¿Qué pasa si... `KEYWORD_CLASSIFICATION_CASE_SENSITIVE` pasa a `true`?
`ECHO` sería `IDENTIFIER`. Una línea como `ECHO hola` podría convertirse en línea huérfana (301), porque la regla exige que la primera palabra no sea reservada.

Escenario 9. ¿Qué pasa si... se agrega `"array"` a `ReservedWords.WORDS`?
`$array = ...` daría 204, y `array(85, 90…)` de codigo.php se clasificaría como KEYWORD (sin error: la línea empieza con `$notas`).

Escenario 10. ¿Qué pasa si... se pone `LexemeValidator` antes de `IdentifierValidator`?
Se pierde la deduplicación: `$ñame = 1;` daría 401 por `ñ` **y** 201. Lo mismo ocurre si se elimina la llamada a `markReported()` en `IdentifierValidator`.

Escenario 11. ¿Qué pasa si... se elimina la recuperación de `StringPattern` (la cadena sin cerrar corre hasta el final)?

La cadena se tragaría el resto del archivo, incluido `?>`: aparecería un 110 falso y no se validaría ninguna asignación posterior.

Escenario 12. ¿Qué pasa si... se elimina la línea `text.contains("<?") ? INLINE : PHP` de `Lexer.tokenize` (siempre INLINE)?

Un archivo sin `<?` sería un solo token `INLINE_TEXT`: solo saldrían 103 y 110 y no se validarían identificadores ni comentarios.

Escenario 13. ¿Qué pasa si... se quitan los tokens WHITESPACE de la lista?

Falla el invariante de lexemas, y el extractor ya no sabría dónde termina el destino: en `$x $y = 1` juntaría `$x$y`. Por eso la regla “mantener espacios y saltos como tokens”.

Escenario 14. ¿Qué pasa si... `CharacterStream.advance` contara `\r` y `\n` por separado?

En archivos CRLF las líneas de los tokens se duplicarían respecto a `SourceLine`; los validadores pedirían líneas inexistentes (`getLine`) y habría excepciones o errores en líneas incorrectas. `valid_codigo_crlf.php` fallaría.

Escenario 15. ¿Qué pasa si... se elimina `isDestructuring`?

`[$p, $q] = [1, 2];` daría un falso 205: el destino crudo `[$p, $q]` tiene base vacía porque empieza con `[`.

Escenario 16. ¿Qué pasa si... se elimina del extractor el salto de un `]` hasta su `[` correspondiente?

`$notas[$i + 1] = 5` se cortaría en el espacio interno y el destino sería `1]`, con 203 falso.

Escenario 17. ¿Qué pasa si... se usa `Character.isLetter` en `AsciiChars.isLetter`?

`ñ` sería letra: `$año` y `$ñame` serían válidos (identificadores Unicode). `latin1.php` dejaría de producir el 202 y fallaría su fixture.

Escenario 18. ¿Qué pasa si... se quita el `stream.reset(afterTagStart)` de `OpenTagPattern`?

Con `<? echo`, los espacios formarían parte del lexema: el 102 mostraría `'<? '`. El invariante sigue cumpliéndose porque nada se pierde.

Escenario 19. ¿Qué pasa si... se elimina el separador extra en `appendErrorSection` cuando la última línea no tiene terminador?

Para un archivo correcto (sin salto tras `?>`), el único separador restante solo cerraría la línea `0053 ?>`: desaparecería la línea en blanco obligatoria antes de los errores. Fallarían los fixtures que terminan sin salto.

Escenario 20. ¿Qué pasa si... se escribe la bitácora siempre en UTF-8?

Un fuente ISO-8859-1 con `ñ` produciría una bitácora cuyos bytes ya no coinciden con el original; `latin1.php` fallaría la verificación de copia byte a byte.

Escenario 21. ¿Qué pasa si... se usa `Files.readAllLines` en lugar de `readAllBytes`?

Se perderían los terminadores originales (CRLF/LF/CR) y la información de si la última línea tenía salto: no se podría detectar el 114 ni copiar byte a byte.

Escenario 22. ¿Qué pasa si... el usuario guarda codigo.php con NetBeans y este agrega un salto final?

Error 114 “1 line break(s) found after the closing tag”. Es comportamiento **correcto** según el PRD.

Escenario 23. ¿Qué pasa si... hay dos archivos .php junto al jar?

Se analizan ambos en orden alfabético y se generan dos bitácoras.

Escenario 24. ¿Qué pasa si... se ejecuta `java -jar compilador.jar codigo.txt`?

Mensaje `ERROR: 'codigo.txt' does not have the required .php extension; the file was not processed.`, no se crea bitácora y el código de salida es 1.

Escenario 25. ¿Qué pasa si... la carpeta es de solo lectura?

`Files.write` lanza `AccessDeniedException`; `Application.analyze` la captura, `printLogWriteError` muestra el motivo y el proceso termina con 1.

Escenario 26. ¿Qué pasa si... se ejecuta desde NetBeans (Run) en vez del jar?

El `CodeSource` es la carpeta `build/classes` (un directorio), así que se usa `user.dir` = raíz del proyecto, donde está `codigo.php`.

Escenario 27. ¿Qué pasa si... se quiere agregar un error nuevo, por ejemplo 206 “identificador de más de 30 caracteres”?

(1) Constante en `ErrorCode` (`IDENT_TOO_LONG(206, Category.IDENTIFIER)`); (2) plantilla `String` en `IdentifierValidator`; (3) condición en `check` usando `name.length()`; (4) fixture y bitácora esperada; (5) documentarlo en INFORME.txt. Nada más cambia: el ordenamiento y la escritura son genéricos.

Escenario 28. ¿Qué pasa si... se quiere reconocer un operador nuevo, por ejemplo `?:`?

Basta agregarlo a `OperatorPattern.OPERATORS`; el orden “más largo primero” se calcula solo. Si necesita un tipo propio, se agrega a la tabla de `TokenClassifier`.

Escenario 29. ¿Qué pasa si... se quisiera validar también `+=` o `.=` como asignaciones?

En `AssignmentTargetExtractor.extract` habría que considerar esos `OTHER_OPERATOR` además de `ASSIGN`, y quitarlos de `BOUNDARY_OPERATORS`. Hoy el PRD pide validar solo `=`.

Escenario 30. ¿Qué pasa si... se reemplazan los autómatas por `java.util.regex`?

Funcionaría, pero **viola** la restricción C3 y la verificación `grep` del proyecto; además se perdería la etapa visible de determinación de patrones que pide el curso.

18. Preguntas probables del profesor (con respuestas)

P1. ¿Qué es un analizador léxico y qué parte del compilador implementaron?

Es la primera fase del compilador: agrupa caracteres en tokens y detecta errores léxicos. Implementamos esa fase para PHP: tokenización completa más validación de etiquetas, identificadores y comentarios.

P2. ¿Cuál es la diferencia entre token, lexema y patrón?

Lexema = texto exacto (`$total`); patrón = regla que lo describe (`\$[A-Za-z0-9_]*`, implementada en `VariablePattern`); token = lexema + tipo + posición (`Token(VARIABLE, "$total", 47, 1)`).

P3. ¿Cuáles son las cuatro etapas y dónde está cada una?

Obtención (`CharacterStream`, `Lexer`), clasificación (`TokenClassifier`, `ReservedWords`, `TokenType`), determinación de patrones (`lexer.pattern`) y validación (`validation`).

P4. ¿Por qué no usaron JFlex o expresiones regulares?

Restricción del proyecto (C2 y C3). Además, escribir los autómatas a mano demuestra la equivalencia entre expresiones regulares y AFD; cada patrón documenta su regex equivalente.

P5. Muéstreme un autómata. ¿Cuáles son sus estados?

`StringPattern`: OPEN → BODY; en BODY la comilla lleva a CLOSED, `\` a ESCAPE y otro carácter a BODY; ESCAPE vuelve a BODY; fin de texto → UNTERMINATED. Método `step` hace una transición por carácter.

P6. ¿Cómo decide el lexer qué patrón usar?

`PatternRegistry.findPhpPattern` prueba en orden fijo y devuelve el primero cuyo `canStart` es verdadero; luego `match` consume el lexema más largo. Si ninguno aplica, `Lexer.matchUnknown` produce UNKNOWN.

P7. ¿Qué es la coincidencia más larga y dónde se aplica?

Elegir el lexema más largo posible. En `OperatorPattern` la lista se ordena por longitud descendente y se toma el primer operador que coincide: `===` antes que `==` antes que `=`.

P8. ¿Por qué importa el orden de los patrones? Dé un ejemplo.

Porque varios pueden empezar en el mismo carácter. `//` debe probarse antes que el operador `/`; `?>` antes que `?`; `.5` (número) antes que `.` (concatenación).

P9. ¿Para qué sirven los modos INLINE y PHP?

Fuera de `<?php` el texto no es código (HTML): en INLINE solo se busca `<?` y lo demás es INLINE_TEXT. `OPEN_TAG` cambia a PHP y `CLOSE_TAG` regresa a INLINE.

P10. ¿Qué pasa si el archivo no tiene `<?php`?

Error 103 en la línea 1, y como recuperación todo el archivo se analiza en modo PHP para que las demás validaciones sigan corriendo.

P11. ¿Cómo encuentran el archivo sin ruta fija?

`PhpFileLocator` obtiene la carpeta del jar con `getCodeSource().getLocation().toURI()`, lista los archivos que terminan exactamente en `.php` y los procesa en orden alfabético. Si no hay, prueba `user.dir`.

P12. ¿Por qué `toURI()` y no `getPath()`?

Porque la URL codifica espacios y tildes (`%20`); convertir con `toURI()` y `Paths.get` decodifica correctamente rutas como `C:\Mis Documentos\compilación`.

P13. ¿Qué pasa con `codigo.PHP` o `codigo.php.txt`?

Se rechazan: la comparación es sensible a mayúsculas y `.php` debe ser la extensión final. Se listan como "Ignored" con el motivo.

P14. ¿Cómo garantizan que no se altera el formato del archivo?

Se leen bytes crudos, cada `SourceLine` guarda su terminador original y la bitácora escribe `NNNN + espacio + contenido + terminador`. `FixtureRunner` verifica byte a byte.

P15. ¿Por qué no usan `readAllLines`?

Porque descarta los terminadores; no sabríamos si eran CRLF, LF o CR, ni si la última línea tenía salto (necesario para el error 114).

P16. ¿Qué es el BOM y cómo lo tratan?

Bytes `EF BB BF` al inicio de algunos UTF-8. Se detecta, se quita del texto, se reporta como 106 y no se copia a la bitácora.

P17. ¿Qué pasa si el archivo no es UTF-8 válido?

El decodificador estricto falla y se usa ISO-8859-1; la bitácora se escribe en el mismo charset para que la copia sea idéntica.

P18. ¿Por qué la bitácora agrega un salto antes de los errores?

Porque un archivo correcto termina en `?>` sin salto; si no se agregara, la línea vacía y los errores quedarían pegados a `?>`. Se usa el primer terminador del archivo como separador.

P19. ¿Por qué guardan los espacios como tokens si no tienen significado?

Por la adyacencia: para extraer el destino de `=` hay que saber qué tokens están pegados; y para el invariante lexemas = texto fuente.

P20. ¿Cómo obtienen el identificador a validar en `$notas[$i] = 5`?

`AssignmentTargetExtractor` retrocede desde `=`, salta espacios, recolecta `]…[` y `$notas` hasta una frontera; el texto crudo es `$notas[$i]` y `getBase()` corta en `[` → se valida `$notas`.

P21. ¿Por qué `$x == 5` no se valida?

Porque `==` es `COMPARISON_OPERATOR`, no `ASSIGN`. Solo el `=` simple activa la validación.

P22. ¿En qué orden se aplican las reglas de identificadores?

205 (vacío), 203 (sin `$`), 201 (`$` solo), 200/201 (primer carácter), 202 (cada carácter posterior), 204 (reservada). Las tres primeras terminan la revisión.

P23. ¿Por qué `$If` no es error pero `$if` sí?

Decisión de diseño: identificadores sensibles a mayúsculas, como las variables de PHP. Está en la constante `IDENTIFIER_CHECK_CASE_SENSITIVE = true`.

P24. Entonces, ¿por qué `ECHO` sí es palabra clave?

Porque en PHP las palabras clave no distinguen mayúsculas; está en `KEYWORD_CLASSIFICATION_CASE_SENSITIVE = false`. Son dos políticas distintas e independientes.

P25. ¿Cómo demuestran que `$total` y `$Total` son distintos?

La tabla de símbolos usa el texto exacto como clave en un `LinkedHashMap`; la consola muestra dos entradas: `$total` (línea 47) y `$Total` (línea 48).

P26. ¿Cómo evitan reportar dos veces el mismo problema?

Con la bandera `reported` del `Token` y el orden fijo de validadores: `IdentifierValidator` marca los tokens de un destino inválido y `LexemeValidator`, que corre al final, los omite.

P27. ¿Cómo se detecta un comentario que ocupa varias líneas con `//`?

No puede: `//` termina en el fin de línea. La siguiente línea, si es texto suelto, es una "línea huérfana"; si la anterior terminaba en comentario se reporta 300, si no 301.

P28. ¿Por qué la división de la línea 22 no da error 302?

Porque 302 solo aplica a una `/` al inicio de la línea o después de `;`, `{`, `}`. En `$suma_notas / $cantidad_notas` la `/` va después de un operando.

P29. ¿Qué pasa con `//` dentro de una cadena?

`StringPattern` consume la cadena completa antes de que el lexer llegue al `//`, así que no es comentario: `"http://example.com # not a comment"` es un solo `STRING_LITERAL`.

P30. ¿Y `?>` dentro de un comentario?

Se ignora porque es parte del lexema del comentario. Si no hay otro `?>`, el error 110 agrega la pista de que el `?>` del comentario fue ignorado. Es una desviación deliberada de PHP real.

P31. ¿Por qué el operador `?->` no cierra el bloque PHP?

`CloseTagPattern` busca exactamente `?>`; `?->` no empieza con `?>`. Lo reconoce `OperatorPattern`.

P32. ¿Qué pasa con una cadena que nunca se cierra?

Error 400; como recuperación la cadena termina al final de su línea (`reset` + `consumeRestOfLine`) y el análisis sigue en la línea siguiente.

P33. ¿Cómo funciona `mark` y `reset`?

`mark` guarda posición, línea, columna y última línea consumida en un objeto `Mark` inmutable; `reset` los restaura. Sirve para retroceder (backtracking) y para extraer el lexema con `textSince`.

P34. ¿Cómo se cuentan las líneas en un archivo CRLF?

En `CharacterStream.advance`, al consumir un `\r` seguido de `\n` solo avanza la columna; la línea se incrementa al consumir el `\n`. Así `\r\n` cuenta como un salto.

P35. ¿Qué garantiza que el lexer no entre en un ciclo infinito?

Todo `match` consume al menos un carácter (`PatternMatch` rechaza lexemas vacíos) y `matchUnknown` siempre consume uno. Por tanto `position` crece en cada vuelta.

P36. ¿Qué es el invariante de lexemas y para qué sirve?

Concatenar todos los lexemas debe dar el texto exacto. Prueba que el lexer no pierde ni duplica caracteres. Lo verifica `LexemeInvariant` en `--tokens` y en `FixtureRunner`.

P37. ¿Cómo ordenan los errores?

`LexicalError implements Comparable`: por línea, luego columna, luego código. `ErrorCollector.getSorted` usa `Collections.sort`.

P38. ¿Por qué `ErrorCode` es un enum?

Centraliza los 29 códigos con su número y categoría, evita números mágicos y errores de tipeo; el compilador verifica que solo se usen códigos existentes.

P39. ¿Cómo probaron el proyecto sin JUnit?

Con `FixtureRunner`, un `main` que analiza 16 archivos de `test-fixtures` y verifica bitácora esperada byte a byte, copia numerada fiel e invariante de lexemas. Imprime PASS/FAIL.

P40. ¿Dónde se aplica la programación orientada a objetos?

Interfaces `LexemePattern` y `Validator` (polimorfismo, patrón Strategy), encapsulamiento con campos privados y `final`, objetos inmutables, enums con datos (`TokenType`, `ErrorCode`), clases anidadas (`Mark`, `SearchResult`), responsabilidad única por clase.

P41. ¿Por qué `Compilador.main` no tiene lógica?

Por separación de responsabilidades y testeabilidad: `Application.run` devuelve el código de salida en vez de cerrar la JVM, así se puede invocar desde otro código.

P42. ¿Qué códigos de salida devuelve y por qué un archivo con errores léxicos devuelve 0?

0 si se analizó al menos un archivo; 1 si no hay archivo, la extensión es incorrecta o hubo error de E/S. Los errores léxicos son el **resultado** del análisis, no un fallo del programa.

P43. ¿Qué es la tabla de símbolos y qué guarda?

Un `LinkedHashMap` nombre exacto → `SymbolEntry` (nombre, primera línea, apariciones), en orden de aparición, construido a partir de los tokens VARIABLE bien formados.

P44. ¿Por qué `$nombre_estudiante` aparece una sola vez si se usa dos veces?

La segunda aparición está dentro de una cadena con comillas dobles; la interpolación está fuera de alcance y la cadena entera es un solo token.

P45. ¿Cómo es compatible con Java 8?

No usa `var`, records, switch expressions, `List.of`, `String.repeat` ni `isBlank`; por ejemplo el `Comparator` de operadores es una clase anónima y `LineText.isBlank` es propio.

P46. ¿Qué hace el target `-post-jar` del build.xml?

Después de crear el jar copia `codigo.php` e `INFORME.txt` a `dist/`, porque `ant clean` borra esa carpeta.

P47. ¿Qué pasaría con un archivo de 12 000 líneas?

Funciona igual; el número se formatea con `%04d`, que no trunca: `12000`. El ancho crece automáticamente.

P48. Si tuvieran que hacer el análisis sintáctico, ¿qué reutilizarían?

La lista de tokens con tipo y posición (filtrando LAYOUT y COMMENT) sería la entrada del parser; el `ErrorCollector` y `LexicalError` podrían generalizarse para errores sintácticos.

P49. ¿Cuál fue la parte más difícil?

Respuesta sugerida: la extracción del destino de la asignación (adyacencia, corchetes, desestructuración, `const`) y las reglas de líneas huérfanas sin falsos positivos (como la división de la línea 22).

19. Hoja de repaso final (memorizar)

En una frase

Programa Java sin librerías que localiza los .php junto al jar, los lee sin alterarlos, los tokeniza con autómatas escritos a mano en cuatro etapas, valida etiquetas, identificadores y comentarios, y escribe una bitácora numerada idéntica al fuente con errores específicos ordenados.

Números clave

Dato	Valor
Líneas / tokens de codigo.php	53 líneas, 274 tokens, 0 errores
Palabras reservadas	20
Códigos de error	29 (100–106, 110–116, 200–205, 300–304, 400–402)
Patrones PHP en el registro	10 (+ OpenTag para INLINE + UNKNOWN como respaldo)
Validadores	5, en orden: Opening, Closing, Identifier, Comment, Lexeme
Fixtures de prueba	16, con 3 verificaciones cada uno
Ancho del número de línea	4 dígitos (`%04d`), separador un espacio
Longitud máxima de un texto citado	40 caracteres + `...`
Códigos de salida	0 éxito / 1 fallo (no encontrado, extensión, E/S)

Orden de patrones (regla mnemotécnica)

“Nuevo Espacio Cierra Comentarios: Bloque, String, Variable, Número, Palabra, Operador” → Newline, Whitespace, CloseTag, InlineComment, BlockComment, String, Variable, Number, Word, Operator.

Los 12 pasos del flujo

1. main → Application.run
2. CliOptions.parse
3. PhpFileLocator (carpeta del jar, `.php` exacto)
4. SourceFileReader (bytes, BOM, UTF-8/ISO, líneas + terminadores)
5. Lexer.tokenize (modo inicial)
6. PatternRegistry → Pattern.match (Etapa 3)
7. TokenClassifier (Etapa 2) + cambio de modo
8. SymbolTable.build
9. ValidationPipeline (5 validadores, Etapa 4)
10. ErrorCollector.getSorted
11. ErrorLogWriter (copia + separador + línea vacía + errores)
12. ConsoleReporter + System.exit

Frases de cierre para la defensa

- “Cada etapa léxica está en su propio paquete y cada clase tiene una sola responsabilidad.”

- “Los patrones son autómatas finitos escritos a mano; su Javadoc indica la expresión regular equivalente.”
- “El archivo nunca se altera: la bitácora es una copia byte a byte, verificada automáticamente.”
- “Se reportan todos los errores, con mensajes específicos, y el análisis nunca se detiene en el primero.”
- “Las decisiones discutibles —mayúsculas, `?>` en comentarios, separador— están en constantes y documentadas.”